

1、微服务简介

1.1、微服务的概念与特征

微服务是指从软件应用中拆分出来的模块，这种模块具有相对独立的业务功能，能被客户程序访问，模块之间也能互相访问

微服务最早由马丁·福勒提出，他们归纳了微服务的基本特征

- 微服务按照业务进行划分
- 微服务运行在独立的进程中
- 微服务采用简单一些通信
- 微服务采用统一的管理框架

1.2、SpringCloudAlibaba框架概述

SpringCloudAlibaba框架的 “全家桶套餐”

微服务注册中心：Nacos

微服务配置中心：Nacos

负载均衡器：LoadBalancer

远程调用框架及组件：Dubbo和OpenFeign

流量控制组件：Sentinel

网关组件：Gateway

消息驱动框架及消息中间件：Stream和RocketMQ

链路追踪组件：SkyWalking

分布式事务管理组件：Seata

1.3、各种软件版本匹配

SpringCloudAlibaba框架依赖SpringCloud和SpringBoot，要求三者的软件版本匹配，才能顺利整合到一起

1.4、云原生的概念

云原生是一种构建和运行应用程序的方法，是一套技术体系和方法论。从字面意思上讲，它包含以下两个部分

- 云：和本地对应，传统的应用运行在本地服务器上，而现在的许多应用会选择运行在云端，及分布式的网络架构中
- 原生：出生时的定位，云和原生组合起来，意为这开发人员在设计开发应用时，就考虑到应用将来会运行在云环境中，会充分运用云资源的优势

云原生可以概括为以下4个技术要点

- 微服务：把软件应用差分成多个微服务。每个微服务都独立部署，这样，当一个微服务出现问题时，其他微服务还能正常对外提供服务
- DevOps：包含开发和运维两个方面，开发和运维不在分开，而是一个整体
- 持续交付：在不影响用户使用服务的前提下，持续把新功能发布给用户使用
- 容器化：容器为微服务提供实施保障，起到对应用的隔离作用。每个服务都能被无差别地部署到容器里，而且容器可以无差别地对其进行管理和维护。

2、helloapp项目

2.1、创建项目结构

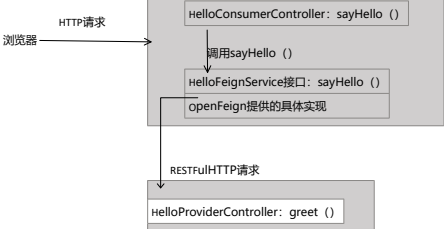
- 2.1.1、创建父工程（外部目录）：创建springboot的父工程，删除src目录，删除pom.xml的依赖和插件，配置springboot、springcloud、springcloudalibaba的版本。
- 2.1.2、再父工程中新建子模块：再父工程目录下新建springboot子模块，新建项目时引入springweb、CloudBootstrap等。再pom.xml文件中引入Nacos等依赖。在application.properties文件中配置Nacos的地址等。
- 2.2、创建微服务hello-provider，在pom.xml引入nacos-discovery。Nacos Discovery组件是hello-provider模块与Nacos服务器通信的桥梁，主要功能如下
- (1) 在启动hello-provider模块时向Nacos服务器注册该模块
 - (2) 从Nacos服务器中订阅所有已经注册的微服务的列表信息



2.2、创建hello-consumer模块，添加SpringWeb、CloudBootstrap、OpenFeign、CloudLoadBalancer依赖

2.2.1、在启动类添加@EnableFeignClients注解

- @EnableFeignClients注解的作用告诉Spring框架，启动时扫描所有用@FeignClient注解表示的FeignClient组件，并把他们注册到Spring框架中。
- HelloConsumerController类通过HelloFeignService接口来访问hello-provider模块的HelloProviderController类。开发人员只需创建HelloFeignService接口，而其具体的实现类由OpenFeign提供。
- 在HelloFeignService类添加@FeignClient（value=“hello-provider-service”），@FeignClient注解指定访问的微服务的名字时hello-provider-service，HelloFeignService接口的实现会根据该微服务名字找到它的网络地址（http://localhost:8081）



2.3、启动微服务的多个实例

- 2.3.1、在“编辑配置”中负责自动配置，创建组合“复合”（添加两个启动配置），通过复合一键启动两个微服务。在Nacos服务列表可以看到运行在两个不同端口微服务
- 2.3.2、HelloConsumerController远程访问hello-provider-service微服务时，无需考虑到底访问它的哪个实例，这是由LoadBalancer负载均衡器决定的。

LoadBalancerAPI中，ReactorLoadBalancer接口表示客户端的负载均衡器，他有两个具体的实现类

- RandomLoadBalancer：采用随机算法，从微服务列表中堆积选择一个微服务实例。
- RoundRobinLoadBalancer：采用轮询算法，从微服务列表中轮流选择一个微服务实例。

2.4、通过restTemplate类访问微服务

- 通过RestTemplate Bean访问hello-provider-service微服务（但是RestTemplate类本身没有整合LoadBalancer）
- 使用LoadBalancerClient接口整合
- 使用@LoadBalanced注解整合

2.4、小结

- SpringCloudAlibaba框架为运行在不同进程中的Spring应该用进行远程通信提供了现成的框架。本范例实现了两个SpringWeb应该你的控制器类之间的通信。这两个SpringWeb应该用属于同一个项目，被调用的一方称为微服务的提供者，调用方称为微服务的消费者。Nacos服务器记录了每个微服务实例的网络地址等信息，消费者通过Nacos服务器获得提供者的地址，再通过OpenFeign组件或者RestTemplate类远程访问提供者的服务
- SpringCloudAlibaba框架的NacosDiscovery等组件的通用步骤：①在pom.xml文件中加依赖；②在application.properties问价那种编写配置代码③编写程序代码

3、微服务的注册与发现

微服务与Nacos服务器之间存在“客户端/服务器”的关系，NacosDiscovery组件运行在客户端，属于客户端组件。微服务通过NacosDiscovery组件与Nacos服务器通信。NacosDiscovery组件会像Nacos服务器注册微服务，也会从Nacos服务器获取微服务列表

3.1、NacosDiscovery的配置属性

配置属性	说明
spring.cloud.nacos.discovery.server-addr	Nacos服务器的网络地址
spring.cloud.nacos.discovery.service	微服务的名字。默认值为应该你的名字{spring.application.name}
Spring.cloud.nacos.discovery.group	微服务所在的分组，默认值为DEFAULT_GROUP
Spring.cloud.nacos.discovery.weight	微服务的权重，取值为1~100.取值越大，权重越高，负载均衡器依据该属性来分配均衡，默认值为1
Spring.cloud.nacos.discovery.network-interface	微服务使用的网卡的名字。如果指定了IP地址，那么默认的网卡是与IP地址对应的网卡；如果没有指定IP地址，那么默认的网卡是主机的第一块网卡
Spring.cloud.nacos.discovery.ip	微服务的IP地址。如果指定了网卡，那么默认的IP地址是与网卡对应的IP地址
Spring.cloud.nacos.discovery.port	微服务的端口，即微服务所在的tomcat服务器监听的端口。可以不用设置，NacosDiscovery组件会自动探测
Spring.cloud.nacos.discovery.namespace	微服务的命名空间的ID。只有同一个命名空间内的为微服务消费者和提供者能够通信。Nacos通过不同的命名空间来区分不同的场景，如开发（develop）和生成（product）场景，确保对不同场景的服务和数据进行隔离
Spring.cloud.nacos.discovery.log-name	日志文件的名字
Spring.cloud.nacos.discovery.endpoint	NacosDiscovery组件向SpringBootActuator提供的被监控的端点的域名，通过此域名可以获得主机的地址

网卡：网卡是一块被设计用来允许计算机在计算机网络上进行通讯的计算机硬件。由于其拥有MAC地址，因此属于OS模型的第1层和第2层之间。它使得用户可以通过电缆或无线相互连接。

Spring.cloud.nacos.discovery.watch-enabled	是否开启nacosDiscovery组件对微服务列表的监控，默认值为true
Spring.cloud.nacos.discovery.watch-delay	监控微服务列表的延迟时间，以毫秒为单位，推荐值为20000ms，即30s
Spring.cloud.nacos.discovery.metadata	根据实际需要设置与注册微服务相关的一些元数据，采用MAP映射格式，如["support.website" : "javathinker.net"]
Spring.cloud.nacos.discovery.register-enable	如果取值为true，当前微服务会向Nacos服务器注册自身。如果为false，当前微服务不会向Nacos服务器注册自身。不管该属性为true或false，当前为微服务都可以作为消费者从Nacos服务器订阅微服务列表，默认值为true
Spring.cloud.nacos.discovery.username	登录Nacos服务器的用户名，默认值为\${spring.cloud.nacos.username}
Spring.cloud.nacos.discovery.password	登录Nacos服务器的口令，默认值为\${spring.cloud.nacos.password}
Spring.cloud.nacos.discovery.heart-beat-interval	心跳间隔时间，以毫秒为单位，推荐为5000ms，即5s
Spring.cloud.nacos.discovery.heart-beat-timeout	心跳超时时间，以毫秒为单位，推荐为15000ms，即15s
Spring.cloud.nacos.discovery.naming-load-cache-at-start	如果取值为true，NacosDiscovery组件会把从Nacos服务器订阅的微服务列表保存到本地文件中，当为微服务重启时，NacosDiscovery组件首先从本地文件中把为微服务列表加载到客户端缓存中，默认值为false
Spring.cloud.nacos.discovery.ephemeral	如果取值为true，表示微服务实例为临时实例；如果取值为false，表示永久实例。默认值为true

3.2、微服务的分组和命名空间

- 3.2.1、在Nacos服务器中注册了许多微服务，为了方便门别类地对这些微服务进行管理，每个微服务都有特定的名字、分组和命名空间，命名空间的范围最大，其次是分组和名字。不同命名空间和分组的微服务允许同名。
- 3.2.2、只有处于同一命名空间以及分组中的微服务才能相互访问。

3.3、微服务的健康检测

3.3.1、Nacos提供了以下两种检测微服务实例健康状态的方式

- 微服务实例主动向Nacos服务器发送心跳
- Nacos服务器反向探测微服务实例的状态

3.3.2、NacosDiscovery组件在为微服务实例级别有一个spring.cloud.nacos.discovery.ephemeral属性，它的默认值为true，表明该微服务实例时临时实例。如果取值为false，则表明该微服务实例时永久实例，也称为持久化实例临时实例和持久化实例的主要区别如下

- 临时实例：主动向Nacos服务器发送心跳，上报健康状态
- 永久实例：Nacos服务器反向探测永久实例的健康状态

3.3.2、微服务的临时实例

Spring.cloud.nacos.discovery.heart-beat-interval: 心跳间隔时间

Spring.cloud.nacos.discovery.heart-beat-timeout: 心跳超时时间

微服务会根据heart-beat-interval指定的间隔时间向Nacos服务器发送心跳，加入该实例宕机，Nacos服务器在heart-beat-timeout设定的时间内没有检测到心跳，就会把该实例的状态设置为不健康，消费者无法再访问它。接下来，再经过heart-beat-timeout设定的时间，如果Nacos服务器还是没有收到微服务实例的心跳，就回答该实例从微服务列表中删除：如果再这段时间内，该实例恢复正常运行，Nacos服务器又收到了它的心跳，那么Nacos服务器会再次把该实例的状态设为健康。

3.3.3、微服务的永久实例

对于永久实例，Nacos服务器会反向检测它的健康状态，对于不健康的永久实例，Nacos服务器允许他们依旧存在微服务列表中，不会将其删除。永久实例具有以下优点

- 运维可以实时了解永久实例的状态
- 便于后续的流控、扩容等操作的稳定实施
- 当保护阈值被触发，起到分流的作用，防止雪崩。（保护阈值是健康实例的比例（当前微服务的健康实例数/当前微服务的实例总数）的临界值。）主要通过以下方式处理雪崩效应
 - 当一个微服务的健康实例的比例小于保护阈值时，意味着健康实例的数目太少，已经无法为所有的消费者提供并发服务，保护阈值就会被触发。
 - 保护阈值被触发后，Nacos服务器就把改为服务的所有实例信息（包括健康实例和不健康实例）全部提供给消费者。消费者有可能会访问到不健康的实例，导致请求失败，但由此造成的损失要比雪崩低很多，因为虽然一部分消费者的请求无法得到正常响应，但是保证了整个系统的可用性
 - 由此可见，讲不健康的永久实例保留再微服务列表中，可以防止产生雪崩。如果临时实例处于不健康状态，Nacos服务器就会将其从微服务列表中删除，因此该临时实例无法再流量洪峰到来时起到分流作用。

4、Nacos服务器配置中心

4.1、微服务的配置属性存放再Nacos服务器中，在Nacos服务器的客户端有一个NacosConfig组件，NacosConfig组件会读取Nacos服务器中的配置属性，再把它们存放在本地的客户端缓存中，微服务的程序代码就能方便地读取缓存中地配置属性

4.2、在Nacos服务器地管理平台中选择“新建配置”，配置DataID、Group、配置格式和配置内容等属性

4.3、在微服务中读取配置属性

4.3.1、在微服务resource目录下新建bootstrap.properties配置文件。NacosConfig组件本身的配置属性必须在bootstrap.properties文件中配置才能生效。并且需要在pom.xml文件中加入springcloud-cloud-starter-bootstrap依赖

4.3.2、Nacos Config组件会把从Nacos服务器中读取到的配置属性绑定到Spring框架的Environment对象中，通过Environment对象的getProperty () 方法读取配置属性。

4.3.3、Spring框架的@Value注解能够把配置属性与一个变量绑定，如

```
@value("${db.username}")
private String username;
```

4.3.4、Spring框架的@ConfigurationProperties注解能够把一组配置属性与一个Java类的一组变量绑定，如

```
@ConfigurationProperties(prefix = "db")
@Component
@Data
public class MyProperties {
    private String username;
    private String password;
}
```

4.4、配置属性的动态更新

- 当Nacos服务器中的配置单元的配置属性发生更新时，NacosConfig组件也会同步更新客户端的相应属性
- NacosConfig组件会在一个长轮询中读取Nacos服务中的配置属性的最新数据。如果确定配置单元中的所有配置属性不会发生更新，可以在bootstrap.properties文件中把spring.cloud.nacos.config.refresh-enable属性设置为false，该属性默认为true。（可以取消NacosConfig组件和Nacos服务器之间的频繁同行）

4.5、不同环境下配置属性的切换

为了便于在不同的环境下切换配置属性，Nacos服务器允许设定与特定环境对应的配置，Nacos Config组件会读取以下配置单元

- DataID为\${application.application.name}的配置单元：默认的配置单元，此配置单元无论在什么环境下都会被NacosConfig组件读取，如hello-config-service
- DataID为\${application.application.name}-\${profile}的配置单元：与特定环境对应的配置单元。如hello-config-service-develop.properties
 - 在不同环境下创建配置单元时，DataID中必须显示指定文件扩展名，否则不能被NacosConfig组件读取，如hello-config-service-develop.properties

4.6、扩展的配置单元

注意事项：

- 在Nacos服务器中创建扩展配置单元时，必须显式指定DataID的文件扩展名
- 默认情况下，NacosConfig组件不会刷新扩展配置单元。如果希望该组件刷新扩展配置单元，则需要把spring.cloud.nacos.config.extension-configs[n].refresh属性设为true，该属性的默认值为false
- 默认配置单元的优先级最高，在所有的扩展配置中，spring.cloud.nacos.config.extension-config[n]中的索引n取值越大，优先级越高。加入这些配置单元中存在相同的配置属性，NacosConfig组件会选用优先级高的配置单元中的配置属性

4.7、监控NacosConfig组件的端点

4.7.1、NacosConfig组件向SpringBootActuator提供了被监控的端点（EndPoint）。在此端点可以一窥NacosConfig组件的相关信息。NacosConfig组件的端点提供以下信息

- NacosConfigProperties: NacosConfig组件的配置属性
 - refreshHistory: 刷新配置属性的历史记录
 - Sources: 从Nacos服务器中读取的所有配置文件
- 4.7.2、以下代码告诉SpringBootActuator，所有的端点都在Web上暴露，既可以通过Web页面来访问各个端点的信息：management.endpoints.web.exposure.include=*

4.8、设置YAML格式的配属性

在Nacos中把配置格式改为YAML，在bootstrap.properties文件中增加spring.cloud.nacos.config.file-extension=yaml

4.9、配置属性的持久化

Nacos服务器对配置属性进行持久化的方式有以下两种

- 默认情况下，Nacos服务器把配置属性保存到内嵌的ApacheDerby数据库中。在Nacos服务器的data/derby-data目录下存放包含配置属性的文件
- 把配置属性报识到外部数据库中，目前支持MySQL数据库，将来可能会支持更多的外部数据库。
 - 在MySQL中，创建名为nacos_config的数据库，执行Nacos服务器的conf/nacos-mysql.sql脚本，创建config_info等数据库表，这些表用于存储配置属性
 - 修改Nacos服务器的conf/application.properties配置文件，设置连接MySQL数据库的信息

4.10、配置属性的回滚

可以在“配置列表”->“更多”->“历史版本”执行回滚。对操作类型为“更新”的历史版本进行回滚，配置单元的内容就会回滚到该历史版本。但是如果对操作类型为“插入”的历史版本进行回滚，配置单元的内容就会被删除

4.11、小结

- 在微服务的bootstrap.properties文件中，只要指定了配置单元的命名空间、分组和DataID，就能和Nacos服务器中的特定配置单元匹配
- 配置单元能在不同环境下切换配置属性，如develop、test、product
- 微服务允许有多个扩展单元，这些扩展单元可以被多个微服务共享

5、Nacos集群

Nacos集群中的一个Nacos节点宕机时，其他Nacos节点会继续工作，保证了集群的可用性。Nacos服务器的startup启动命令有三种运行模式。

运行模式	启动命令	说明
单机模式	Startup -m standalone	默认使用内嵌Derby数据库，也可以配置外部数据库
集群运行+内嵌Derby数据库	Startup -p embedded	使用内嵌Derby数据库
集群运行+MySQL数据库	startup	必须配置外部MySQL数据库

在实际的生产环境中，对于Nacos集群所访问的MySQL数据库，建议搭建采用主从模式的MySQL数据库集群，这样能提高数据库系统的可用性

5.1. Nacos集群的Raft算法

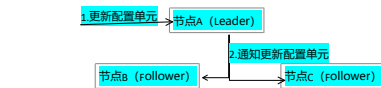
5.1.1. Raft算法要求集群中节点的数目为奇数，并且至少要有三个节点。每个节点充当以下三种角色之一

- Leader（领导）：协调每个接待你的数据同步，定时向Follower发送心跳，表明自己活着，任何时候，集群中都只有一个Leader。
- Follower（随从）：负责响应Leader和Candidate的请求
- Candidate（竞选人）：这是过渡角色。当集群中的Leader宕机后，Follower就有可能变成Candidate，发起新一轮的选举

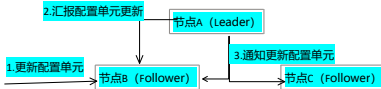
5.1.2. 节点之间数据的同步

假设有三个Nacos节点：节点A、节点B和节点C。节点A为Leader，节点B和节点C为Follower。节点之间数据的同步分为以下两种情况

- (1) 如果节点A上的配置单元发生了更新，节点A作为Leader，会直接通知节点B和节点C同步更新配置单元，如图。



- (2) 如果节点B上的配置单元发生了更新，节点B作为Follower，会先向节点A汇报配置单元发生更新，再通知所有的Follower同步更新配置单元，如图



5.1.3. 节点的选举机制

为了保证选举顺利举行，Raft算法做了以下限定

- 收到过半票数的Candidate成为Leader
- 当一个Candidate被告知其他节点已经称为Leader时，则自己切换为Follower
- 当一个Candidate在一段时间内没有说到过半的投票数，并且集群内尚未出现Leader时，该Candidate会重新发起新一轮的选举

5.2 搭建Nacos集群

5.2.1. 配置Nacos节点

在单机模式部署Nacos集群（以三个为例）+MySQL数据库

- 在MySQL中，创建名为nacos_config的数据库，执行Nacos服务器的conf/nacos-mysql.sql脚本，创建config_info等数据库表，这些表用于存储配置属性
- 修改Nacos服务器的conf/application.properties配置文件，设置连接MySQL数据库源的信息
- 在cluster.conf文件中加入三个Nacos节点的地址信息
- 修改Nacos服务器的conf/application.properties配置文件，设置端口信息
- 根据主机实际内存，修改startup.cmd的Java虚拟机的内存参数。
set "NACOS_JVM_OPTS=-server-Xms256m -Xmx256m -Xmn128m"
-Xms: 初始分配内存，默认值为2g，改为256m
-Xmx: 内存最大值，默认值为2g，改为256m
-Xmn: 新生代内存，默认值为1g，改为128m
- 负责Nacos目录，总共三份，在application.properties文件中修改端口

5.2.3. 避免Nacos节点之间的端口冲突

Nacos2一共监听4个端口：Nacos版本2支持gRPC通信协议，针对gRPC协议新增了两个端口。假定Nacos服务器的server.port为8810，那么他的新增端口为9810、9811。新增端口相对于基本端口8810的偏移量分别为+1000和+1001。Raft协议的7810，相对基础端口8810偏移量为-1000 所以为了防止不同Nacos节点的新增端口冲突，每个Nacos节点server.port属性的取值最好间隔大一些



5.2.4. 通过微服务访问Nacos集群

在bootstrap.properties文件中做如下修改。spring.cloud.nacos.discovery.server-addr用于指定Nacos服务器的网络地址，spring.cloud.nacos.config.server-addr用于指定该组件所访问的服务器的地址。

```
spring.cloud.nacos.discovery.server-addr=\
127.0.0.1:8810,127.0.0.1:8820,127.0.0.1:8830
spring.cloud.nacos.config.server-addr=\
127.0.0.1:8810,127.0.0.1:8820,127.0.0.1:8830
```

5.3. Nacos集群的AP和CP运行模式

在分布式系统中，有一个重要的CAP理论，他为管理分布式的节点提出了三个约束条件

- C (consistency)：强一致性。集群中每个节点的数据始终保持一致
- A (Availability)：可用性。集群系统一直处于可用状态
- P (Partition Tolerance)：分区容忍性。当集群中由于网络通信故障而割裂成多个区域时，这些区域的节点仍然可以被正常访问

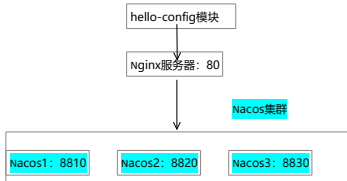
对于分布式系统，分区容忍性P时不可或缺的，如果客户比较在意能得到及时的响应，那么AP时首选。如果客户要求数据不能有半点差错，那么CP是首选

5.4. 使用Nginx反向代理服务器

- (1) Nginx是高性能的HTTP服务器和反向代理服务器，能够保证负载均衡

当代理服务器为客户端提供代理，访问目标服务器时，称为正向代理；当代理服务器为服务端提供代理，向目标客户提供服务时，称为反向代理。

- (2) 把Nacos集群和Nginx服务器整合后，hello-config微服务不在直接访问Nacos1、Nacos2和Nacos3节点，而是访问Nginx服务器，又Nginx服务器把请求转发给集群中的Nacos节点



- (3) 再conf/nginx.conf文件中配置需要代理的Nacos节点的地址，以及代理转发的URL。

在http模块中添加如下内容，表明当用户请求访问的URL为http://localhost:80/nacos/，Nginx接收该请求后再转发给http://nacos-cluster/nacos/所对应的Nacos集群中的节点

```
upstream nacos-cluster{
    server 127.0.0.1:8810;
    server 127.0.0.1:8820;
    server 127.0.0.1:8830;
}

server{
    listen 80;
    server_name localhost;

    #指定代理转发url:
    location /nacos/ {
        proxy_pass http://nacos-cluster/nacos/;
    }
}
```

- (4) 在hello-config的bootstrap.properties文件中，把原先的Nacos服务器的地址改为Nginx的服务器的地址

5.5. Linux服务器部署keepalived+nginx+nacos集群

- (1) 克隆三台ubuntu



- (2) 下载nacos(依赖jdk)

- ```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export PATH=$PATH:$JAVA_HOME/bin
```

```
echo $JAVA_HOME #验证配置
```

- ```
172.26.86.120:6610
172.26.86.129:6610
172.26.86.130:6610
```

下載nginx: `sudo apt install nginx -y`

- ```
upstream nacos_cluster{
```

验证nginx配置是否有语法错误: `sudo nginx -t`

```
Sudo apt install keepalived -y
```

- ```
global defs {
    notification_email {
        acassen
    }
    notification_email from Alexandre.Cassen@firewall.lac
    smtp_server 172.26.86.128
    smtp_connect_timeout 30
    router_id %s%}

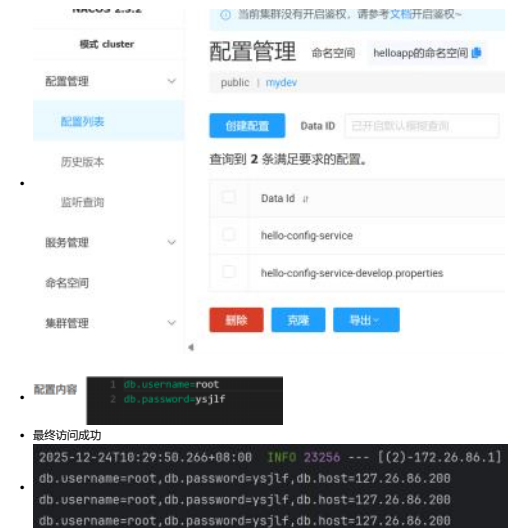
wrap_script chk.nginxw{
    #在脚本中调用nginxw命令是否可运行的脚本
    script /etc/keepalived/nginx_check.sh"
    interval 2 #脚本执行的时间，为2s
    weight 2 #权重，如果这个脚本的检查结果为true，则权重高权重高
}

wrap_instance V1.1 {
    state MASTER
    interface ens33
    warn_master_delay 10
    smtp_alert
    virtual_router_id 51
    priority 102
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass 1234
    }
    track_script {
        chk.nginx
    }
    virtual_ipaddress {
        172.26.86.200/24

        # optional label, should be of the form "realdev:isontext" for
        # compatibility with isconfd"
    }
}
```

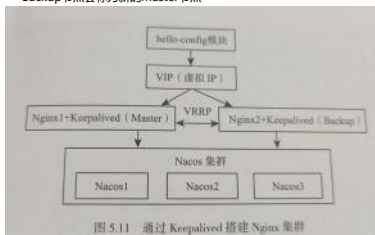
- ```
A=ps -C nginx --no-header | wc -l
if [$A -eq 0];then
/usr/sbin/nginx
sleep 2
B=ps -C nginx --no-header | wc -l
if [$B -eq 0];then
systemctl stop keepalived
fi
fi
exit 0
```

- ← → 🔍 ⚠️ 不安全 172.26.86.200:8848/nacos/#/clusterManagement?serverId=center&group=&data=



## 5.5. 小结

- (1) 对于Nacos集群，每个Nacos节点通过Raft协议进行紧密的通信。通过选举产生一个Leader和多个Follower，有Leader协调每个节点上数据的同步
- (2) 对于Nginx集群，每个Nginx节点通过Keepalived进行协调，Keepalived检测Nginx节点的状态，从存活的节点中选出Master和Backup
- (3) 在Nacos集群中，Nacos节点负责注册和维护为服务列表，以及维护配置单元，Nacos集群会保证各个Nacos节点上各种数据的一致。
- (4) 在通过Keepalived简历的Nginx集群中，Nginx节点主要负责把请求转发给Nacos集群中的节点。Nginx节点不需要处理业务逻辑，也不需要存储与微服务相关的数据。Nginx集群只需要保证可用性就可以了。当Master节点宕机后，Backup节点会称为新的Master节点



## 6. 远程调用OpenFeign

### 6.1. OpenFeigh是Spring开源组织开发的远程调用组件，是微服务消费者与提供者通信的桥梁

- @FeignClient注解的value属性或者name属性指定所访问的微服务的名字，如
- ```
@FeignClient("hello-provider-service")
public interface HelloFeignService {
```
- 为了提高程序代码的可维护性，可以在配置文件中指定所访问的微服务的名字，如在application.yaml文件中定义一个name配置属性
- ```
provider:
 name: hello-provider-service
```
- ```
@FeignClient("${provider.name}")
public interface HelloFeignService {
```

6.2. 优化设定映射URL的根路径

如果接口中所有服务方法的映射URL中都有相同的根路径，可以在@FeignClient注解的path属性中设定映射URL的根路径

6.3. 输出详细日志

6.3.1. 为了便于查找通信失败的原因，可以输出OpenFeign产生的日志。OpenFeign产生的日志分为4种级别

日志级别	说明	性能和使用场合
NONE	没有任何日志，这是默认值	性能最佳，适用于生产环境
BASIC	仅记录请求方法、URL、响应转码代码以及执行时间	性能中等，适用于生产环境，便于了解通信过程和通信性能
HEADERS	在记录BASIC级别日志的基础上，还记录请求头和响应头	性能中等，适用于开发以及测试环境，便于调试程序
FULL	记录请求头、请求正文、响应头、响应正文，以及元数据等，信息最全面	性能最低，应用于开发以及测试环境，便于调试程序

6.3.2. SpringBoot把应用程序的日志分为6种级别，由低到高依次如下：

TARCE<DEBUG<INFO<WARN<ERROR<FATAL

6.4. 请求数据和响应数据的压缩

6.4.1. OpenFeign没有开启对请求数据和响应数据的压缩。为了提高网络传输的效率，可以开启对传输数据的gzip压缩功能，需要设置OpenFeign的以下属性

```
• feign.compression.request.enabled: 当属性值为 true 时，开启对请求数据的压缩功能，默认值为 false。
• feign.compression.request.enabled: 当属性值为 true 时，开启对响应数据的压缩功能，默认值为 false。
• feign.compression.request.accept-gzip: 指定待压缩的请求数据的类型。
• feign.compression.request.accept-gzip-size: 指定待压缩的请求数据的临界大小。例如，如果数据为 2048，则表示当请求数据的大小超过 2048 字节时，就对其进行压缩；如果小于 2048 字节，就不会对其运行压缩。
```

为了提高网络传输效率和减轻CPU运行负荷之间进行平衡，可以把feign.compression.request.min-request-size属性设置得大一点，避免对小规模请求数据进行压缩，减少压缩次数。这种折中措施既能减轻CPU运行负荷，又能保证网络传输效率。

6.5. 超时配置

OpenFeign的以下两个配置属性分别用于设定连接超时时间和读取数据超时时间

- connectTimeout属性：连接微服务提供者的超时时间，以毫秒为单位
- readTimeout属性：读取来自微服务提供者的响应数据的超时时间，以毫秒为单位

```
client:
  config:
    default:
      connect-timeout: 1000
      read-timeout: 5000
```

6.6. 异常处理

当消费者访问提供者时，如果出现读取数据的超时错误，会返回对用户很不友好的错误信息，让用户难以理解。为了向用户提供更好的错误提示页面，可以运用OpenFeign的回调功能。

OpenFeign的回调功能有以下两种实现方式

- (1) 创建实现OpenFeign服务接口的回调类
- (2) 创建实现OpenFeign服务接口的回调类和工厂类。

OpenFeign的回调功能依赖于Sentinel流量控制器

6.1. OpenFeign与Sentinel的整合

- 在pom.xml中加入Sentinel的依赖

```
<dependency>
  <groupId>com.alibaba.cloud</groupId>
  <artifactId>spring-cloud-starter-alibaba-sentinel</artifactId>
</dependency>
</dependencies>
```

- 在application.yaml中把feign.sentinel.enabled属性设为true，开启Sentinel对OpenFeign的流量控制

```
feign:
  sentinel:
    enabled: true
```

6.6.2. HelloFeignFallback类作为回调类，实现了HelloFeignService接口。如果访问hello-provider-service微服务出现异常，OpenFeign组件就会调用HelloFeignFallback类的相关方法。

- (1) 创建HelloFeignFallback类，它实现了HelloFeignService接口中的所有方法，负责处理异常，返回对用户友好的提示信息。
- (2) 在HelloFeignService接口的@FeignClient注解中通过fallback属性指定回调类，如fallback=HelloFeignFallback.class

6.6.3. 创建回调类和工厂类

HelloFeignFallback回调类无法知道异常的详细信息，因此不能有针对性地处理异常。为了把具体地异常信息传给HelloFeignFallback回调类，可以创建HelloFeignFallback类地工厂类。

- (1) 在HelloFeignFallback回调类中添加Throwable类型地cause属性，表示当前地异常

```
public class HelloFeignFallback implements HelloFeignService {
    private Throwable cause; // 2个用法
    public HelloFeignFallback() {}
    public HelloFeignFallback(Throwable cause){
        this.cause=cause;
    }
}
```

- (2) 创建回调类地工厂类HelloFeignFallbackFactory，该类实现了OpenFeignAPI中地FallbackFactory接口，它地create () 方法负责创建HelloFeignFallback类地实例，会把当前地异常对象传给HelloFeignback实例

```
@Component
public class HelloFeignFallbackFactory implements FallbackFactory<HelloFeignFallback> {
    @Override
    public HelloFeignFallback create(Throwable cause) {
        return new HelloFeignFallback(cause);
    }
}
```

- (3) 在HelloFeignService接口的@FeignClient注解中通过fallbackFactory属性指定回调类的工厂类

```
@FeignClient(value = "${provider.name}", configuration = HelloFeignConfig.class,
    fallbackFactory = HelloFeignFallbackFactory.class)
public interface HelloFeignService {
    @GetMapping("/{greet}/{username}") // 1个实现
    String sayHello(@PathVariable("username") String username);
}
```

最终是：HelloFeignService请求hello-provider-service——>产生异常，OpenFeign调用HelloFeignFallbackFactory工厂类——>工厂类把异常传给HelloFeignFallback回调类，回调类返回友好提示

6.7. 传递对象参数

除了String类型，OpenFeign和微服务提供者这还可以传递简单对象和复杂对象

简单对象：对象的属性为基本类型、基本类型的包装类型，以及String类型

复杂对象：对象的属性不仅可以为基本类型、基本类型的包装类型，以及String类型，还可以为简单对象类型或复杂对象类型

在hello-common中创建3个对象类型

```
Entity
  Name
  Result
  User
```

在hello-provider模块和hello-consumer模块的pom.xml文件中，都需要加入对hello-common模块的依赖，这样就能访问Name类、User类和Result类。如下

```
<dependency>
  <groupId>demo</groupId>
  <artifactId>hello-common</artifactId>
  <version>0.0.1-SNAPSHOT</version>
</dependency>
```

在HelloProviderController类中创建testName () 方法，name参数用@SpringQueryMap注解标识，表明该参数是简单对象类型。

```
@GetMapping("/{user/{testname}")
public Result testName(@SpringQueryMap Name name) {...}
```

在HelloProvider类中创建testUser () 方法，user参数用@RequestBody注解标识，表明该参数是复杂对象类型。

```
@PostMapping("/{user/{testuser}")
public Result testUser(@RequestBody User user) {...}
```

总体流程：浏览器访问hello-consum的controller的方法——>controller的方法调用HelloFeignService接口的方法——>HelloFeignService接口的方法由OpenFeign远程调用hello-provider的对应方法

如果调用helloprovider的方法发生异常，OpenFeign会通过HelloFeignFallback工厂类调用HelloFeignFallback回调方法，返回友好的错误提示。

6.8. 小结

消费者和提供者之间的底层是通过HTTP协议进行通信的，而OpenFeign封装了HTTP通信细节，是的消费者可以按照RPC的凡是远程访问提供者所提供的微服务。

7. 远程调用框架：pubbo

7.1. 比较Dubbo和OpenFeign

(1) Dubbo是框架，而OpenFeign是组件。Dubbo作为远程调用框架，需要同时部署到微服务的提供者和消费者中。OpenFeign仅仅是消费方的远程调用组件。

(2) Dubbo提供了真正的RPC远程调用，而OpenFeign实际上仅仅提供了伪RPC调用。

(3) Dubbo在进行远程调用时，消费者与提供者之间采用专门的dubbo协议进行通信，而OpenFeign与提供者之间采用HTTP协议进行通信。HTTP协议要求消费者发送的请求数据中包含庞大的请求头。

要求提供者发送的响应数据中包含庞大的响应头，而dubbo协议则不需要通信双方发送许多额外数据，因此dubbo协议是轻量级的通信协议，比HTTP协议具有更好的通信性能。

7.2. 创建采用dubbo框架的范例

7.2.1. 在本范例中，消费者和提供者有一个共同的服务接口HelloService，它位于hello-common公共模块中。

```
public interface HelloService { // 4个用法
    public String sayHello(String username);
}
```

在hello-provider的pom.xml文件中引入dubbo依赖。

springCloudAlibaba2022版本之前：

```
<dependency>-->
  <groupId>com.alibaba.cloud</groupId>-->
  <artifactId>spring-cloud-starter-alibaba-nacos-discovery</artifactId>
</dependency>-->
<dependency>-->
  <groupId>com.alibaba.cloud</groupId>-->
  <artifactId>spring-cloud-starter-dubbo</artifactId>-->
</dependency>-->
```

SpringCloudAlibaba2022版本之后：dubbo-registry-nacos: Dubbo 对接 Nacos 注册中心的核心依赖。dubbo-spring-boot-starter: Dubbo 与 Spring Boot 的整合启动器

```
<dependency>
  <groupId>org.apache.dubbo</groupId>
  <artifactId>dubbo-registry-nacos</artifactId>
</dependency>
<dependency>
  <groupId>org.apache.dubbo</groupId>
  <artifactId>dubbo-spring-boot-starter</artifactId>
</dependency>
```

7.2.2. 修改hello-provider的application.yml文件

```
dubbo:
  application:
    name: hello-provider-service
  registry:
    address: nacos://172.26.86.128:8848
    username: nacos
    password: nacos
    parameters:
      namespace: 6ee54f4f-25fb-46cf-af23-9218e9d7cdc2
      group: MY_GROUP
  scan:
    base-packages: demo.helloprovider
  protocol:
    name: dubbo
    port: -1
```

7.2.3. 在启动类添加@EnableDubbo

```
@EnableDubbo
@SpringBootApplication
public class HelloProviderApplication {
```

7.2.4. hello-consumer微服务同理

7.2.5. nacos配置配置中心

服务列表

命名空间: helloapp的命名空间

public | mydev

创建服务 服务名称: 请输入服务名称 分组名称: 请输入分组名称 隐藏空服务: ☒ 查询

服务名	分组名称	集群数目	实例数	健康实例数	触发保护阈值	操作
hello-provider-service	MY_GROUP	1	1	1	false	详情 示例代码 订阅者 删除
providers.demo.hellocommon.service.HelloService:	MY_GROUP	1	1	1	false	详情 示例代码 订阅者 删除

7.2.6. 访问http://localhost:8082/enter/Tom

localhost:8082/enter/Tom

Hello,Tom

7.3. 超时时间和重试次数设置

可以在提供者方、消费者方、全局访问以及特定服务类设置超时时间和重试次数。如果由冲突，dubbot会按照以下规则选用优先级高的配置

(1) 针对特定服务类的配置的优先级高于全局范围的配置

(2) 针对消费者方的配置的优先级高于提供者方的优先级。

注：在提供者方设置超时时间和重试次数比较合适，因为提供者方作为服务的提供方，更清楚以何种方式访问服务具有很好的性能。

如在hello-provider的HelloServiceImpl中添加超时时间。

```
@DubboService(timeout = 3000)
public class HelloServiceImpl implements HelloService {
    @Override
    public String sayHello(String username) {
        try {
            Thread.sleep(10000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        return "Hello,"+username;
    }
}
```

localhost:8082/enter/Tom

Whitelabel Error Page

this application has no explicit mapping for /error, so you are seeing this as a fallback.

Thu Dec 25 10:21:54 CST 2025

there was an unexpected error (type=Internal Server Error, status=500).

7.4. 异常处理

在7.3节中，当消费者远程访问提供者出现超时错误时，消费者的程序代码没有处理这种错误，而是完全由服务器来处理。如果消费者希望自己处理错误，只需在调用helloService接口时捕获异常即可。

```
@GetMapping("/{username}")
public String sayHello(@PathVariable String username) {
    try {
        return helloService.sayHello(username);
    } catch (Exception e) {
        return username+",something is wrong."
            + "<br>" + getStackTrace(e);
    }
}
```

localhost:8082/enter/Tom

Tom,something is wrong.

org.apache.dubbo.rpc.RpcException: Failed to invoke the met

7.5. 传递复杂对象类型数据

Dubbo对需要在网上传输的复杂对象类型进行了一个约束，要求他必须实现Serializable接口。

7.6. 负载均衡

dubbo在远程访问hello-provider-service微服务时，如果该微服务由多个实例，则Dubbo会保证负载均衡。Dubbo支持以下5中负载均衡算法。

• random: 随机算法。随机算法选择微服务的实例，权重高的实例获得更多被选中的机会。这是Dubbo的默认负载均衡算法。

- roudrobin：轮询算法。轮流来选择微服务的所有实例，权重高的实例获得更多被选中的机会。
- consistenthash：一致性哈希算法。根据哈希值选择微服务实例，参数相同的请求总是访问同一个实例。
- leasactive：最少活跃调用算法。首先选择最不活跃的微服务实例，如果每个实例的活跃度相同，就随机选择一个实例
- shorrestresponse：最短响应算法，这是Dubbo2.7版本以上新增的算法。选择响应时间最短的微服务实例。如果每个实例的响应时间相同，就随机选择一个实例。

如果Dubbo的负载均衡算法（通过loadbalance属性、weight属性设置）在提供方、消费方、全局方位以及针对特定服务类都设置了负载均衡算法，Dubbo会按照以下规则选用优先级高的配置。

- (1) 针对特定服务类的配置优先级高于全局范围的配置。
- (2) 针对消费者方的配置的优先级高于提供者方的配置。通常在提供方设置默认的负载均衡算法，而消费者方根据实际需要可以覆盖提供者方的负载均衡算法。

7.7、Dubbo与Sentinel的整合

Sentinel是流量控制组件，在hello-provider-service整合Dubbo和Sentinel的步骤如下：

- (1) 在hello-provider的pom.xml中添加以下依赖

```
<dependency>
  <groupId>com.alibaba.cloud</groupId>
  <artifactId>spring-cloud-starter-alibaba-sentinel</artifactId>
</dependency>
<dependency>
  <groupId>com.alibaba.csp</groupId>
  <artifactId>sentinel-apache-dubbo-adapter</artifactId>
  <version>1.8.4</version>
</dependency>
<dependency>
  <groupId>com.alibaba.csp</groupId>
  <artifactId>sentinel-transport-simple-http</artifactId>
</dependency>
```

- (2) 在hello-provider的application.yml中配置sentinel控制台的地址

```
cloud:
  sentinel:
    transport:
      dashboard: 127.0.0.1:8080
```

7.8、提供者回调消费者

在Dubbo框架中，消费者可以远程调用提供者的服务方法。此外，提供者也可以远程调用消费者的方法，这种调用过程称为回调

图7.14展示了整个helloapp软件项目的类框架图。其中，HelloService接口和CallbackListener接口位于hello-common模块，MyCallbackListener类和HelloConsumerController类位于hello-consumer模块，HelloServiceImpl类位于hello-provider模块。

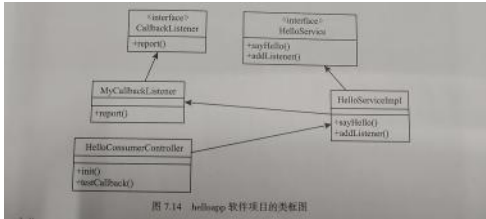


图 7.14 helloapp 软件项目的类框架图

图7.15展示了消费者和提供者中的类互相通信的时序图

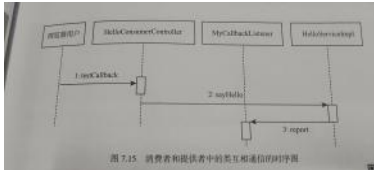


图 7.15 消费者和提供者中的类互相通信的时序图

7.9、消费者异步调用提供者的服务方法

Dubbo框架提供了异步调用机制。如果提供者的一个服务方法要执行很长时间，而消费者在发出调用后，不希望进入阻塞状态一直等待响应结果，而是能继续执行后续的其他业务，等待提供者的服务方法有了返回结果后再进行处理。

入图7.16，当消费者异步调用提供者的doTask（）方法时，会立即返回null，消费者继续执行后续的操作。等到提供者执行完doTask（）方法，再把真正的响应结果发送给消费者，消费者再处理响应结果。

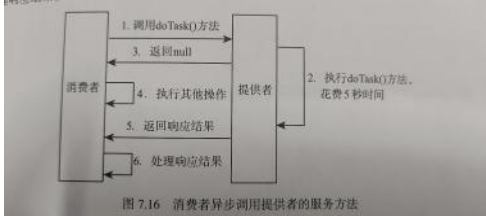


图 7.16 消费者异步调用提供者的服务方法

核心方法：在hello-consumer微服务，通过getCompletableFuture获得 Dubbo 异步 RPC 调用的结果载体，在whenComplete核心方法完成回调逻辑

```
CompletableFuture<String> helloFuture=
    RpcContext.getContext().getCompletableFuture();
helloFuture.whenComplete((String result,Throwable exception)->{
    if(exception==null){
        System.out.println(result);
    }else {
        exception.printStackTrace();
    }
});
```

7.10、小结

- OpenFeign使用的是很广泛的超文本传输协议。HTTP协议按照传输网络数据的思路来设计，消费者发送请求数据，提供者返回响应数据，该协议不是专门为RPC设计的；而Dubbo默认使用定制的轻量级dubbo协议，在进行远程调用时具有更好的运行性能。Dubbo时专门为RPC设计的，消费者通过调用服务接口来访问服务，提供者通过实现服务接口来提供服务。
- dubbo局限性：dubbo协议不适合传输大数据包的服务调用

8、流量控制组件：Sentinel

本章中消费者可以是微服务或者浏览器。有了OpenFeign和Dubbo还不能保证消费者和提供者之间顺利快速地通信，还取决于以下因素

- 消费者与提供者之间地网络通畅，没有断开
- 提供者没有宕机，处于可用状态
- 请求数量没有超过提供者地承载负荷，确保提供者能够对每个请求及时做出响应

8.1、微服务容错的基本原理

8.1.1、为了给消费者提供更友好地服务，可以运用流量控制组件sentinel，它能提高微服务系统的容错能力。

8.1.2、在微服务中，某个节点出故障或者请求流量迎来洪峰，都是不可控的。因此，流量控制组件提供了以下三种应对突发状况的措施。

- (1) 限流：避免请求流量一起涌进来，冲垮系统。对超过阈值的请求拒绝响应。
- (2) 隔离：对访问特定服务的工作线程的数目进行限制。
避免雪崩：在图8.5中，服务A调用共服务B和服务C，如果服务B出现故障宕机，引发雪崩，导致服务A也宕机，那么会使服务A无法再访问服务B的工作线程数目。例如，阈值为10，表示服务A最多只允许10个工作线程访问服务B，当10个工作线程都处于阻塞时，服务A不会再增加新的线程访问服务B，服务B实际上就被隔离了，这样就会避免服务A节点由于无限制地增加访问服务B地线程而被击垮，从而避免雪崩

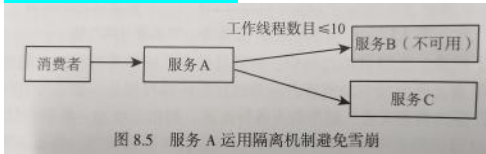


图 8.5 服务 A 运用隔离机制避免雪崩

(3) 熔断与降级：禁止访问已经出现故障或者存在安全隐患的节点。熔断的后果是导致整个分布式系统降级运行，因为部分节点不再提供服务。例如，只有基础服务仍然在运行，暂时不允许访问一些负载高但是不太重要的不稳定服务。

8.2、微服务与Sentinel的整合

在微服务内部，需要集成Sentinel控制器，sentinel控制器也称为sentinel客户端。在微服务外面，有一个独立的sentinel控制台，用户可以通过浏览器访问sentinel控制台，就能控制微服务的流量

在spring cloud alibaba中，为了把微服务与其他软件整合，经常会运用这样的思路：在微服务中安装一个插件，由这个插件与软件主程序进行通信。

8.2.1、在hello-consumer的pom.xml中引入Sentinel依赖

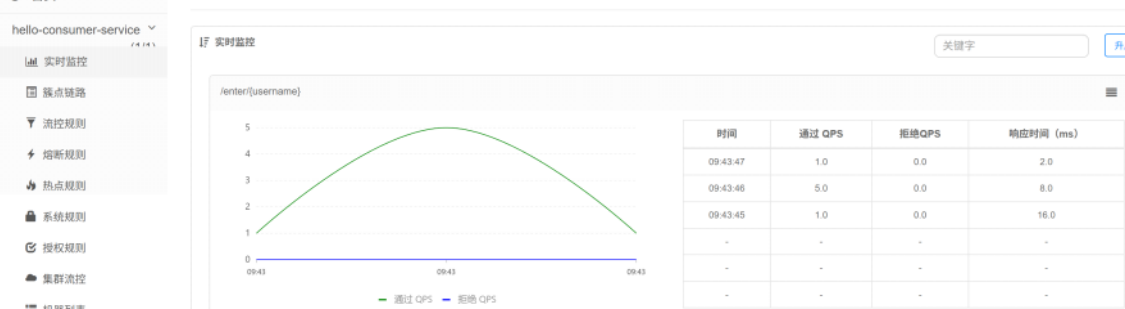
```
<dependency>
  <groupId>com.alibaba.cloud</groupId>
  <artifactId>spring-cloud-starter-alibaba-sentinel</artifactId>
</dependency>
```

在hello-consumer的application.yaml中配置sentinel控制台的地址

```
cloud:
  sentinel:
    transport:
      dashboard: 127.0.0.1:8080
```

8.2.2、安装sentinel控制台，下载sentinel-dashboard-1.8.*-jar文件，启动sentinel

8.2.3、访问hello-consumer的/enter(/username)就可以看大流量信息。



8.3、流控规则

sentinel控制台流量地方式有以下两种

- (1) 限流：限定QPS（每秒请求/查询数量）
- (2) 隔离：限定并发地工作线程地数目

8.3.1、流控模式

流控模式指定当某资源地限流统计项达到阈值时，就对当前资源进行流控。该模式有以下三个可选值

- 直接：当前资源地限流统计项达到阈值时，对当前资源限流。
- 关联：关联资源地限流统计项达到阈值时，对当前资源限流
- 链路：调用链路上的入口资源的限流统计项达到阈值时，对当前资源限流

8.3.2、流控效果

流控效果分为快速失败、warm Up（预热）和排队等待

- 快速失败：当请求被限流时，会立即返回拒绝页面。
- warm Up：Warm Up就是给微服务提供一个缓冲时间，单机阈值再一段时间内从默认的初始值3增加到指定的值。
- 排队等待：排队等待是把不规则的请求流量调整为匀速的流量，这样能保证服务允许地处理请求。

8.4、熔断规则

在微服务地调用链路中，当一个微服务出现不稳定地情况时，为了避免局部地不稳定导致整个微服务系统地雪崩，可以对不稳定微服务的调用。微服务的不稳定包括以下两种情况

- 响应请求的时间过长
- 在单位时间内产生大量异常

熔断规则根据不稳定情况的类型有以下三种策略可选。

- 慢调用比例：在单位时间内，响应时间过长（慢调用）的请求数目占所有请求数目的比例。
- 异常比例：在单位时间内，响应结果为异常的请求数目占所有请求数目的比例
- 异常数：在单位时间内，响应结果为异常的请求数目。

ruleType有以下可选值
flow：标识流控规则
degrade：标识熔断规则
system：标识系统规则

8.5、系统规则

流控规则和熔断规则都是针对微服务的某个资源设置的，而系统规则是针对整个微服务设置的规则，用于控制微服务的入口流量。

流控规则的“阈值类型”有5个可选值：LOAD、RT、线程数、入口QPS、CPU使用率a

8.6、授权规则

- 授权规则根据请求的调用方来判断通过或拒绝请求。
- 授权规则包括指定白名单和黑名单两种方式
- 授权规则中的“流控应用”配置项用于指定调用者的来源名单，以英文逗号隔开。

8.7、@SentinelResource注解

前面介绍的各种规则都是通用的，如果还需要针对特定资源定制客户化规则，则可以使用@SentinelResource注解，它具有以下属性

- (1) value属性：指定资源的名字
- (2) blockHandler属性：指定当请求被Sentinel拒绝时的处理方法
- (3) blockHandlerClass属性：指定当请求被Sentinel拒绝时的处理类
- (4) fallback属性：指定响应过程中产生的异常的处理方法
- (5) fallbackClass属性：指定响应过程中产生的异常的处理类
- (6) exceptionsToIgnore属性：只需要被忽略的异常。当出现这些异常时，fallback属性和fallbackClass属性的设定值不起作用。

当控制器类的一个请求处理方法用@SentinelResource注解标识时，它成为@Sentinel的资源，可以为它指定客户化的规则，主要包括以下三个方面

- (1) 指定热点规则，针对热点参数进行限流
- (2) 指定当请求被sentinel拒绝时的处理方法。
- (3) 指定响应过程中产生的异常的处理方式

8.8、自定义处理BlockException异常的方式

sentinel依照各种规则控制请求流量，在拒绝一个请求时，会产生BlockException异常，更确切地说，是产生BlockException类地特定子类异常，具体包括以下类型。

- FlowException：违反流控规则产生地流控异常
- ParamFlowException：违反热点规则产生的热点异常
- DegradException：违反熔断规则产生的熔断降级异常
- AuthorityException：准放授权规则产生的授权异常
- SystemBlockException：违反系统规则产生的系统异常

8.9、集群流控

在实际运行环境中，到达集群中的每个节点请求流量并不均匀，而所有的实例节点都遵循同样的单机流控规则，就会出现一种不合理的情况：当集群的总流量还没到达阈值时，有的节点就已经限流了。因此，需要从集群的角度进行整体限流，并以单机限流兜底，更好地发挥流量控制地效果。

8.9.1、集群流控地原理

集群限流由Token Server负责统计集群的总流量。专门用来统计就请你的总流量的节点称作Token Server，其他节点都是Token Client，如图8.34所示，Token Client接收到一个请求时，回向Token Client接收到一个请求时，会向Token Server申请Token，如果能获得Token，就说明当前的QPS还未达到集群的总阈值，允许通过当前请求；如果不能获得Token，就说明当前的QPS已经达到集群的总阈值，当前请求被拒绝。

8.9.2、集群流控中的节点的身份有以下两种

- Token Client：集群流控客户端，每接收到一个请求，就会试向Token Server申请Token。
- Token Server：集群流控服务器，处理来自Token Client的t申请，根据配置的集群流控规则判断是否因该发放Token（是否允许通过当前请求）。

8.9.3、sentinel集群流控支持两种规则：限流规则和热点规则，并支持以下两种总阈值计算模式

- 集群总体模式：直接指定集群的QPS的总阈值
- 单机均摊模式：指定单机均摊的阈值。Token Server会根据所有节点的数目来计算总阈值。

8.10、sentinel规则的持久化

sentinel控制台的管理页面上配置的各种规则存在于内存中，当终止sentinel控制台时，配置的规则就随着内存的回收而消失。所以可以把规则存放放到Nacos配置中心，让sentinel从Nacos配置中心加载规则

8.10.1、流控规则的持久化

在hello-consumer-service中的pom.xml文件中引入依赖

```
<dependency>
  <groupId>com.alibaba.csp</groupId>
  <artifactId>sentinel-datasource-nacos</artifactId>
</dependency>
```

在Nacos控制台添加sentinel的配置

配置内容

命名空间 a71212c7-136f-4f79-8ba2-b5c2441811cd

Data ID hello-consumer-ds1

Group DEFAULT_GROUP

更多配置选项

描述

MDS: a4ec1d5d7e7e23470d53eb8e8a1445

配置内容

```
1 {
2   {
3     "resource": "/enter/{username}",
4     "limitApp": "default",
5     "grade": 1,
6     "count": 1,
7     "strategy": 0,
8     "controlBehavior": 0,
9     "clusterMode": false
10  }
11 }
```

在application.yml配置连接Nacos控制台的信息

datasource:

ds1:

nacos:

server-addr: 127.0.0.1:8810

data-id: hello-consumer-ds1

group-id: DEFAULT_GROUP

namespace: a71212c7-136f-4f79-8ba2-b5c2441811cd

rule-type: flow

8.11. 小结

- 当请求流量的洪峰到来时，sentinel会进行限流，即限制通过的请求数目。加入被请求的资源不稳定，响应时间过长或者频繁抛出异常，该请求就会熔断，暂时不允许访问该资源。
- sentinel既可以对微服务的特定资源设点控制规则，又可以对微服务的入口流量设定系统规则，还可以对微服务集群进行流控。对于特定的资源，还可以通过@SentinelResource注解设定客户化的热点参数规则，以及定制处理异常的方式。

9. 网关组件：GateWay

GateWay是由SpringCloud组织开发的网关组件，如图9.1所示。GateWay会浴池里所有访问微服务系统的请求，然后按照既定的路由，把请求转发给相应的微服务。



图 9.1 GateWay 会预处理所有的请求

Gateway面向最终用户，接受的是HTTP请求。对于微服务系统内部由消费者微服务向提供者微服务发出的请求，则不有Gateway进行转发。

9.1. Gateway简介

GateWay按照响应式编程的模式开发，运用了Spring的WebFlux响应式框架，以高性能的Netty作为服务器。GateWay包括以下三大核心技术。

- 路由转发：按照既定的路由，把请求转发给相应的微服务
- 断言：判断请求是否符合特定条件
- 过滤：在请求到达微服务之前，以及微服务返回相应结果之后，进行的一些通用处理。

9.2. 创建网关服务模块

创建一个集成了GateWay组件的hello-gateway模块，以提供网关服务

(1) 在hello-gateway模块的pom.xml中添加gateway依赖

<dependency>

<groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-gateway</artifactId>

</dependency>

(2) 在hello-gateway的application.yml中配置hello-provider和hello-consumer

spring:

application:

name: hello-gateway-service

cloud:

gateway:

routes:

- id: provider-route

uri:

http://localhost:8081

order: 1

#路由权重值，数值越小，优先级越高，默认为0

predicates:

- Path=/greet/**

#路径的匹配条件，**为通配符

- id: consumer-route

uri:

http://localhost:8082

order: 1

predicates:

- Path=/enter/**

server:

port: 88

id：路由id，任意设置，只要保证唯一即可

uri：路由转发的URL地址

order：指定路由的优先级，数值越小，优先级越高，默认为0，当两个路由的

Path属性相同时，优先级高的路由会优先匹配

predicates：指定匹配条件/路径/断言，通过配置了匹配条件/断言

(3) Gateway首先接收访问的HTTP请求，然后根据路由配置中的路径的匹配条件，把这两个请求分别转发给hello-provider-service微服务二号hello-consumer-service微服务

9.3. GateWay与Nacos的整合

如果微服务的实例改变了监听端口，或者一个微服务有多个实例，那么以上设置方式会很死。为了更灵活地设置路由，可以把GateWay与Nacos整合。在GateWay地路由配置中，只需把URL设置为微服务的名字，与微服务名字对应的具体地址则由Nacos提供。

(1) 在hello-gateway模块引入Nacos和LoadBalancer的依赖

<dependency>

<groupId>com.alibaba.cloud</groupId>

<artifactId>spring-cloud-starter-alibaba-nacos-discovery</artifactId>

</dependency>

<dependency>

<groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-loadbalancer</artifactId>

</dependency>

由于GateWay组件运行在Netty服务器中，而不是运行在Tomcat中，因此在pom.xml文件中不能加入spring-boot-starter-web依赖，否则在启动hello-gateway模块时会报错

(2) 在hello-gateway的pom.xml中配置LoadBalancer、GateWay和Nacos

cloud:

loadbalancer:

enabled: true

gateway:

discovery:

locator:

enabled: true

routes:

- id: provider-route

uri:

lb://hello-provider-service

order: 1

#路由权重值，数值越小，优先级越高，默认为0

predicates:

```
cloud:
  loadbalancer:
    enabled: true
  gateway:
    discovery:
      locator:
        enabled: true
    routes:
      - id: provider-route
        uri:
          lb://hello-provider-service
        order: 1
        predicates:
          - Path=/greet/** #路径匹配规则, 微服务小, 优先被选中, 默认匹配
      - id: consumer-route
        uri:
          lb://hello-consumer-service
        order: 1
        predicates:
          - Path=/enter/**
  nacos:
    discovery:
      server-addr: 127.0.0.1:8848
```

(3) Gateway说先接收访问URL的HTTP请求, 然后根据路由配置中的路径的匹配条件, 把两个请求分贝转发给hello-provider-service微服务和hello-consumer-service微服务。

9.4、断言

断言用于指定匹配路由的判断条件, 只有请求符合特定的判断条件, Gateway才会根据匹配的路由, 把请求转发到相应的微服务

9.4.1、内置的断言工厂类

(1) 基于日期时间类型的断言工厂类。
基于日期时间类型的断言工厂类根据日期时间进行判断, 主要有以下三个。
* AfterRoutePredicateFactory: 接收一个日期时间参数, 判断请求的日期时间是否晚于指定的参数值。

• AfterRoutePredicateFactory: 接收一个日期时间参数, 判断请求的日期时间是否早于指定的参数值。
• BetweenRoutePredicateFactory: 接收两个日期时间参数, 判断请求的日期时间是否在指定的时间范围内。
示例代码如下:
判断请求的日期时间是否在指定的日期范围内
- after: 2024-07-15T11:23:34.743Z-2024-07-15T11:23:34.743Z
判断请求的日期时间是否在指定的日期范围内
- between: 2024-07-15T11:23:34.743Z-2024-07-15T11:23:34.743Z
(2) 基于主机名的断言工厂类: HostRoutePredicateFactory。
HostRoutePredicateFactory 接收一个表示期望匹配的主机名参数, 判断发出请求的主机地址是否在指定的地址范围内。示例代码如下:
- host: 192.168.1.1
(3) 基于 Cookie 的断言工厂类: CookieRoutePredicateFactory。
CookieRoutePredicateFactory 接收两个参数: Cookie 名称和正则表达式, 判断请求中是否含有指定名称的 Cookie, 并且其 Cookie 的值是否与给定的正则表达式匹配。示例代码如下:
判断请求是否含有名为 cookie 的 Cookie, 并且其 Cookie 的值是否为小写字母
- cookie: cookie, [a-z]*
(4) 基于 HTTP 请求头的断言工厂类: HeaderRoutePredicateFactory。
HeaderRoutePredicateFactory 接收两个参数: HTTP 请求头名称和正则表达式, 判断 HTTP 请求头中是否含有指定名称的键, 并且该键的值是否与给定的正则表达式匹配。示例代码如下:
判断请求头中是否含有名为 x-forwarded-for 的键, 并且其值为 IP 地址
- header: x-forwarded-for, [a-z]*
(5) 基于远程主机名的断言工厂类: RemoteHostRoutePredicateFactory。
RemoteHostRoutePredicateFactory 接收一个参数, 表示期望匹配的主机名, 判断发出请求的主机地址是否与给定的主机名匹配。示例代码如下:
- remoteHost: 192.168.1.1
(6) 基于 HTTP 请求方式的断言工厂类: MethodRoutePredicateFactory。

基于 HTTP 请求方式的断言工厂类: MethodRoutePredicateFactory。
MethodRoutePredicateFactory 接收一个参数, 表示期望匹配的 HTTP 请求方式。
HTTP 请求方式包括 GET、POST、PUT 和 DELETE 等。示例代码如下:
- method: GET
(7) 基于 Path 请求路由的断言工厂类: PathRoutePredicateFactory。
PathRoutePredicateFactory 接收一个参数, 表示期望匹配的 Path 模式, 判断请求的 URI 是否与给定的 Path 模式匹配。示例代码如下:
- path: /greet/**
(8) 基于 Query 请求参数的断言工厂类: QueryRoutePredicateFactory。
QueryRoutePredicateFactory 接收四个参数: 请求参数名称、正则表达式、判断请求是否含有指定名称的请求参数, 并且该请求参数的值是否与给定的正则表达式匹配。示例代码如下:
判断请求是否含有名为 query 的请求参数, 并且其值为 true
- query: query, true
(9) 基于权重路由的断言工厂类: WeightRoutePredicateFactory。
WeightRoutePredicateFactory 接收四个参数: 权重名称、权重值、对同一个组内的路由按照权重进行排序。示例代码如下:
权重路由
- weight: weight, 1
权重路由
- weight: weight, 2
以上代码配置了两个路由, route1 和 route2, 它们的 Path 匹配规则相同, 都是 /greet/**, 但是 route1 的权重为 1, route2 的权重为 2。因此, 对于路径为 /greet/** 的请求, route1 的权重为 1, route2 的权重为 2。因此, 对于路径为 /greet/** 的请求, route1 的权重为 1, route2 的权重为 2。因此, 对于路径为 /greet/** 的请求, route1 的权重为 1, route2 的权重为 2。

9.4.2、自定义断言工厂类

Gateway还允许开发人员创建自定义断言工厂类, 以设定对请求的客户化判断条件。自定义断言工厂类只要继承AbstractRoutePredicateFactory类即可
为hello-gateway模块创建了自定义断言工厂类AgeRoutePredicateFactory, 它创建的Age断言接收两个参数: 年龄最小值和年龄最大值。Age断言的判断条件是请求参数age的取值在给定的年龄最小值和年龄最大值之间。

9.5、过滤器

Gateway运用过滤器对HTTP请求进行预处理, 以及对生成的响应结果进行附加处理。过滤器分为全局过滤器和局部过滤器。全局过滤器对所有的路由都起作用, 而局部过滤器只对特定的路由起作用。Gateway提供了内置的过滤器, 还允许开发人员自定义过滤器。

9.5.1、局部内置过滤器

9.5.2、默认过滤器

9.5.3、全局过滤器

9.5.4、自定义局部过滤器

Gateway允许开发人员创建自定义局部过滤器工厂类, 设定客户化的过滤操作。在Gateway API中有一个AbstractGatewayFilterFactory接口, 它为GatewayFilterFactory接口提供了基础的实现。自定义局部过滤器工厂类只要继承AbstractGatewayFilterFactory接口即可。
在hello-gateway模块中自定义PringParamGatewayFilterFactory, 它创建的过滤器PrintParam有一个param参数, 用于指定请求参数的名字。PrintParam过滤器会在控制台中输出指定的请求参数的值。

```

@Component
public class PrintParamGatewayFilterFactory extends AbstractGatewayFilterFactory<PrintParamGatewayFilterFactory> {
    public PrintParamGatewayFilterFactory() {
        super(Config.class);
    }
    // 0个用法
    @Override
    public List<String> shortcutFieldOrder() {
        return Arrays.asList("param");
    }
    // 1个用法
    @Override
    public GatewayFilter apply(Config config) {
        return ((ServerWebExchange exchange, GatewayFilterChain chain) -> {
            ServerHttpRequest request = exchange.getRequest();
            if (request.getQueryParams().containsKey(config.param)) {
                request.getQueryParams().forEach((String v) -> {
                    System.out.println("请求参数: " + config.param + "=" + v);
                });
            }
            return chain.filter(exchange);
        });
    }
    @Data
    public static class Config {
        private String param;
    }
}

```

通过浏览器访问http://localhost/enter/Tom?age=21后，hello-gateway模块的控制台会输出“请求参数age=21”



9.5.5、自定义全局过滤器

如果要让自定义全局过滤器对所有的路由都起作用，则需要实现Gateway API的GlobalFilter接口和Ordered接口。

在hello-gateway中自定义一个全局过滤器，它会拒绝不包含请求参数flag的所有请求

```

@Component
@Order
public class FlagGatewayFilter implements GlobalFilter, Ordered {
    // 0个用法
    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        log.info("FlagGatewayFilter: new Date()");
        String flag = exchange.getRequest().getHeader("flag");
        if (flag == null) {
            log.info("FlagGatewayFilter: flag is null, not allow request");
            exchange.getResponse().setStatusCode(HttpStatus.NOT_ACCEPTABLE);
            byte[] response = "Access Refused: " + exchange.getRequest().getPath().getBytes();
            DataBuffer wrapResponse = exchange.getResponse().wrap(response);
            return exchange.getResponse().writeWith(
                Flux.just(wrapResponse));
        }
        return chain.filter(exchange);
    }
    // 1个用法
    @Override
    public int getOrder() {
        return 0;
    }
}

```

9.6、GateWay与sentinel的整合

(1) 在hello-gateway中，为了把GateWay与Sentinel整合，需要引入它们的网关适配器依赖

```

<dependency>
    <groupId>com.alibaba.csp</groupId>
    <artifactId>sentinel-spring-cloud-gateway-adapter</artifactId>
</dependency>

```

(2) 只需要创建一个用@Configuration注解标识的配置类，就可以在hello-gateway模块中设定网关限流。具体操作如下：

- 初始化一个SentinelGatewayFilter类型的Bean组件，以全局过滤器的身份对请求流量进行限流
- 初始化一个SentinelGatewayBlockExceptionHandler类型的Bean组件，处理BlockException异常，也就是处理请求被Sentinel拒绝的情况
- 创建表示网关限流规则的GatewayFlowRule对象，并通过GatewayRuleManager.loadRules()方法加载这些限流对象。

9.6.1、对路由限流

在hello-gateway模块的application.yaml文件中，配置了一个ID为provider-route的路由。在RouteGatewayConfiguration类为provider-route路由设置了限流规则：1s内请求数目的限流阈值为1，并且只有当请求中包含参数age时，才会使用该限流规则。

9.6.2、对自定义的API入口限流

ApiDefinition类表示自定义的API入口，它包含一个或多个URL路径。

在hello-gateway的ApiGatewayConfiguration类定义两个API入口

- 第一个API入口：名字为consumer-service-api，包含的路径为/enter/**和/list
- 第二个API入口：名字为provider-service-api，包含的路径为/greet/**

```

// 初始化自定义API入口
// 1个用法
private void initGatewayRules() {
    Set<GatewayFlowRule> rules = new HashSet<>();
    rules.add(new GatewayFlowRule(RESOURCE_PROVIDER_SERVICE_API)
        .setResourceMode(SentinelGatewayConstants.RESOURCE_MODE_CUSTOM_API_NAME)
        .setCount(1)
        .setIntervalSec(1));
    rules.add(new GatewayFlowRule(RESOURCE_CONSUMER_SERVICE_API)
        .setResourceMode(SentinelGatewayConstants.RESOURCE_MODE_CUSTOM_API_NAME)
        .setCount(1)
        .setIntervalSec(1));
    GatewayRuleManager.loadRules(rules);
}

// 初始化自定义API入口
// 1个用法
private void initCustomizedApis() {
    Set<ApiDefinition> definitions = new HashSet<>();
    ApiDefinition api1 = new ApiDefinition(ApiName.CONSUMER_SERVICE_API)
        .setPredicateItems(new HashSet<>() {
            add(new ApiPathPredicateItem()
                .setPattern("/enter/**")
                .setMatchStrategy(SentinelGatewayConstants.URL_MATCH_STRATEGY_PREFIX));
            add(new ApiPathPredicateItem()
                .setPattern("/list"));
        });
    definitions.add(api1);
}

```

9.6.3、同时对路由和API入口限流

可以同时使用ApiDefinition和GatewayFlowRule分别对API和路由设置限流规则

9.7、跨域配置

同域访问：如果前端代码与后端代码都存放在同一个服务器上，则前端代码向服务器发送请求称为同域访问（存放在服务器上的前端代码被下载到浏览器端，浏览器执行前端代码。当浏览器端的前端代码试图向服务器发出请求时，浏览器认为这是同域访问，会直接放行）。

跨域访问：如果前端代码与后端代码存放在不同的服务器上，则前端代码向服务器发送请求称为跨域访问。如图9.6中，存放在服务器A上的前端代码被下载到浏览器端。当浏览器端的前端代码试图向服务器B发出请求时，浏览器认为这是跨域访问。默认情况下，许多浏览器禁止这种跨域访问，会提示违反了CORS（跨域资源访问）政策。

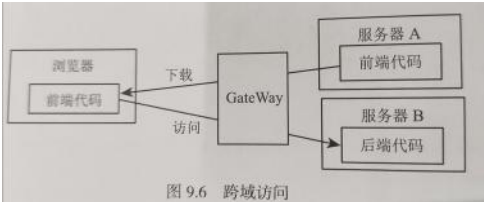


图 9.6 跨域访问

GateWay 支持以配置方式指定跨域访问条件，满足条件的请求可以进行跨域访问。如下，在 hello-gateway 模块的 application.yaml 文件中，用于设置跨域访问条件

```
gateway:
  globalcors:
    cors-configurations:
      '[/**]':
        # 适用所有的路径
        allowed-origins: # 允许跨域访问的客户端来源的域名，* 表示所有
          - "*"
        allowed-methods: # 允许的 HTTP 请求方式
          - "*"
        allowed-headers: # 允许的 HTTP 请求头
          - "*"

```

9.8. 超时配置

GateWay 的以下两个属性用于可以用于设置全局超时时间和局部超时时间

connect-timeout: 连接超时时间，以毫秒为单位

response-timeout: 响应超时时间，可以显示指定时间单位，如 5s。

9.9. 通过 Actuator 监控网关

在 hello-gateway 模块中，通过 Spring Boot 的 Actuator 监控网关的配置

(1) 引入 Actuator 的依赖

(2) 在 application.yaml 中配置以下属性，management.endpoint.gateway.enabled 设置为 true，表示允许 GateWay 向 Actuator 暴露端点；management.endpoints.web.exposure.include 属性设置为 gateway，表示允许 GateWay 通过 Web 页面访问 GateWay 的端点。

```
management:
  endpoint:
    gateway:
      enabled: true
  endpoints:
    web:
      exposure:
        include: 'gateway'

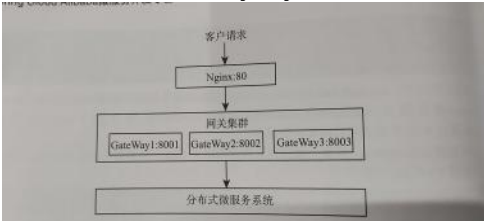
```

通过浏览器访问 GateWay 的路由信息



9.10. 网关集群

图 9.8 展示了网管集群架构。客户请求首先到达 Nginx，Nginx 根据特定的负载均衡策略，把请求发送给网关集群的一个 GateWay 节点，GateWay 节点再把请求转发给相应的微服务。



步骤：

(1) 为 hello-gateway 模块的创建三个启动配置，监听不同端口

(2) 修改 Nginx 安装目录下的 conf/nginx.conf 文件，增加需要代理的 GateWay 节点的地址，以及代理转发的 URL。

9.11. 小结

网关是从外网进入内网的入口，对由多个微服务节点构成的内网起保护作用。网关具有以下功能

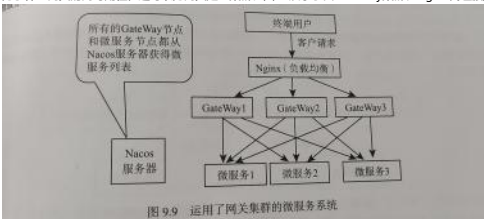
(1) 路由转发：为了保护内网服务的安全，通常不会暴露内网中微服务节点的地址，而是暴露网关地址，并通过路由建立网关地址和内网地址的对应关系。

(2) 流量控制：对进入网关的请求流量限流，防止流量洪峰击垮内网。

(3) 负载均衡：按照特定负载均衡策略，把请求转发给某个微服务节点。

(4) 过滤请求：对请求进行预处理，以及对响应结果进行附加处理。

为了保证网关的高可用性，还可以为网关建立集群。图 9.9 展示了由 GateWay 集群、Nginx 代理服务器、Nacos 服务器及微服务节点组成的微服务系统。



10. 消息驱动框架：Stream

对于两个微服务之间的通信，数据的传输也分为同步模式和异步模式

同步模式：微服务 A 请求访问微服务 B 时，微服务 A 先发出请求，微服务 B 会立即接收到请求。同样，当微服务 B 返回响应结果时，微服务 A 会立即接收到响应结果

异步通信：把微服务 A 与微服务 B 之间发送的请求或响应一律称为消息，消息由消息中间件同意接收和分发。（当微服务 A 通过消息中间件向微服务 B 发送消息时，微服务 A 是消息的生产者，微服务 B 就是消息的消费者；反之亦然）

10.1. 消息中间件简介

• 消息中间件把消息存放在消息队列中，生产者向消息队列发送消息，消费者从消息队列中接收消息。

• 消息队列采用“先进先出”的数据结构，确保生产者发送消息的顺序和消费者接受消息的顺序一致

10.1.1. 消息中间件常用于分布式微服务系统的应用解耦、异步处理、流量削峰等场景。

1. 应用解耦

软件系统的耦合性越高，容错性就越低，而同步通信会增加微服务之间的耦合度。利用消息中间件能对微服务系统解耦

2、异步处理

如果有些操作耗时很长，并且不需要同步处理，那么采用异步处理方式能提高微服务的响应效率

3、流量削峰

消息中间件还能起到控制流量的作用。

削峰：如图10.8所示，当请求流量高峰到来时，如果1s内出现了5000个请求，那么支付管理微服务在1s内只能处理1000个请求，其余的4000个请求存放在消息队列中。由此可见，消息中间件把高峰“削”掉了。

填谷：等到流量高峰过后，支付管理微服务消费消息的熟练仍然维持在1000QPS，逐渐消费队列中积压的消息，这个过程称为“填谷”

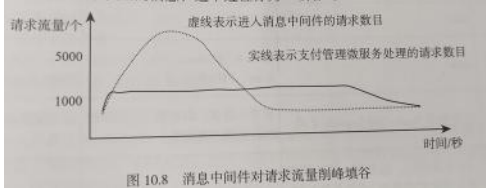


图 10.8 消息中间件对请求流量削峰填谷

10.1.2、消息中间件的缺点

消息中间件主要有以下三个缺点

- 系统可用性降低：微服务系统引入的外部依赖越多，出错环节也越多。一旦消息中间件宕机，整个系统就无法正常运转
- 系统稳定性降低：消息中间件必须保证消息没有被重复消费，也没有丢失，以及按照先后顺序传递消息。如果由于异常等因素不能保证这些需求，就会降低系统的稳定性，影响业务顺利完成。
- 分布式一致性问题：假设一个任务需要微服务A、微服务B和微服务C异步执行，共同完成，当微服务A执行成功时，返回了响应结果，而微服务B和微服务C并没有完成自己的子任务，这会导致业务数据的不一致

10.1.3、消息中间件产品

比较信息	消息中间件		
	RocketMQ	kafaka	RabbitMQ
所属组织/公司	由阿里巴巴公司开发，已经加入Apache开源组织	Apache开源组织	Rabbit公司
开发语言	Java	Scala	Erlang
客户端语言	Java	C/C++、Python、Go、Erlang、Ruby、PHP等	Java、C/C++、Python、PHP、Perl等
控制台	官方提供RocketMQ Console工具	官方未提供，但有第三方开源控制台工具可供使用	官方提供RabbitMQ Admin工具
优点	(1) 在高吞吐、低延迟、高可用方面有非常好的表现，当消息堆积时，性能也很好。 (2) 支持多种消费方式 (3) 支持BrokerServer对消息过滤 (4) 支持事务消息 (5) 支持消息的顺序消费，消费者可以水平扩展，消费能力强 (6) 集群规模在50台左右；单日处理消息达上百亿条；经过真实场景中大数据流量的考验，比较稳定	(1) 在高吞吐、低延迟、高可用、集群热扩散、集群容错方面有非常好的表现 (2) 为生产者提供缓存、压缩功能，提高了性能和效率 (3) 支持消息的顺序消费 (4) 支持多种客户端语言 (5) 生态完善，在大数据处理方面有大量配套软件支撑	(1) 在高吞吐、高可用不如前两者 (2) 支持多种客户端语言 (3) 具有很好的性能 (4) 管理界面很丰富，在互联网公司有较大规模的运用
缺点	(1) 生态不够完善，处理消息堆积和吞吐量的能力低于Kafka (2) 对于集群，不支持主从自动切换，Master宕机后，消费者要过一段时间才能感知 (3) 客户端只支持Java	(1) 消费者集群数目受到分区数目的限制 (2) 单机Topic很多时，性能会明显下降 (3) 不支持事务消息	(1) Erlang语言难度较大，集群不支持动态扩展。 (2) 不支持事务消息，消息吞吐量有限 (3) 消息堆积时，性能会明显下降

10.2、RocketMQ简介

RocketMQ是由阿里巴巴公司开发的开源消息中间件，支持事务消息、顺序消息、批量消息和定时消息。与其他消息中间件相比，RocketMQ具有以下优势。

- 支持事务消息，消息发送和数据库操作会保证最终一致性
- 通过RocketMQ通信的多个系统的数据会保证最终一致性
- 支持18个级别的延迟消息
- 支持按照指定次数和时间间隔重新尝试消费失败消息
- 支持消费者端的消息过滤，减少不必要的网络传输
- 允许消费者的重复消费

10.2.1、RocketMQ涉及的基本概念

RocketMQ传送消息涉及以下基本概念：消息生产者、消息消费者、主题、代理服务器、名字服务器、拉去式消费、推送式消费、生产者组、消费者组、集群消费、广播消费、消息和标签

10.2.2、RocketMQ的消息收发模型

图10.9展示了RocketMQ的消息收发模型。其中，生产者负责生产消息；消费者负责消费消息；BrokerServer负责存储和转发消息。每个BrokerServer可以存储多个主题的消息，每个主题的消息也可以分片存储在集群的多个BrokerServer实例中。

NameServer的作用与Nacos有些类似，都是提供服务注册与发现的功能。NameServer会注册哥哥BrokerServer节点，监控BrokerServer节点的实时状态，为生产者和消费者提供每个主题的路由信息



图 10.9 RocketMQ 的消息收发模型

10.2.3、安装和启动RocketMQ

下载和解压RocketMQ的binary包，在Windows中配置以下两个环境变量

- ROCKETMQ_HOME: RocketMQ的安装根目录
- NAMESRV_ADDR: NameServer的地址，设置为默认值local host:9876

环境变量配置好后，在RocketMQ的bin目录下，运行以下两个命令文件

Mqnamesrv.cmd: 启动NameServer

Mqbroker.cmd: 启动BrokerServer

10.2.4、安装和启动RocketMQ控制台

下载RocketMQ控制台，用IDEA打开，修改application.properties的rocketmq.conf.namesrvAddr属性，指定NameServer的地址

```
rocketmq.conf.namesrvAddr=127.0.0.1:9876
```

10.2、搭建RocketMQ集群

为了提高RocketMQ的可用性，可以建立NameServer和BrokerServer集群。

- NameServer集群：NameServer是几乎无状态的节点，可以集群部署，节点之间无任何信息同步
- BrokerServer集群：集群部署相对复杂，BrokerServer节点按角色分为Master与Slave。
 - 一个Master可以对应多个Slave，但一个Slave只能对应一个Master
 - Master与Slave的对应关系通过相同的Broker Name和不同的Broker ID来设定，Broker ID为0表示Master，非0表示Slave
 - 每个BrokerServer节点与NameServer集群中的所有节点建立长连接，定时向所有NameServer节点注册主题信息
- Producer集群：Producer与NameServer集群中的一个节点（随机选择）建立长连接，定期从NameServer获取主题路由信息。
 - Producer与作为Master的一个BrokerServer节点建立长连接，并且定时向该BrokerServer节点发送心跳。
 - Producer完全无状态，可以集群部署
- Consumer集群：consumer与NameServer集群中的一个节点（随机选择）建立长连接，定期从NameServer获取主题路由信息。
 - consumer与作为Master或Slave的一个BrokerServer节点建立长连接，并且定时向该BrokerServer节点发送心跳。
 - consumer既可以从Master接收消息，也可以从Slave接收消息

producer向Master发送的消息存放在Master的队列中，该消息还会通过异步复制或同步双写的方式保存到Slave中。Producer不能向Slave发送消息，而Consumer可以从Master或Slave中接收消息

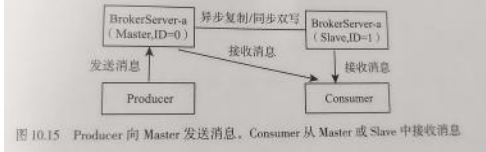


图 10.15 Producer 向 Master 发送消息。Consumer 从 Master 或 Slave 中接收消息

10.3、搭建BrokerServer集群

在RocketMQ的conf目录下有三个子目录，他们分别按照不同的模式配置BrokerServer集群。

- zm-noslave子目录：多Master模式
- 2m-s-async子目录：多Master多Slave的异步复制模式
- 2m-s-sync子目录：多Master多Slave的同步双写模式

10.3.1、多Master模式

在多Master模式下，BrokerServer集群中无Slave，全是Master。

多Master模式的优缺点如下：

- 优点：配置简单，单个Master节点宕机或重启对应用无影响。
- 缺点：在单个节点宕机期间，该节点上未被消费的消息在节点恢复之前不可订阅，消息实时性会受到影响

10.3.2、多Master多Slave的异步复制模式

- 在多Master多Slave的异步复制模式下，每个Master对应一个或多个Slave，有多组Master/Slave。
- 写到Master中的消息会异步复制到Slave中，Master和Slave之间有毫秒级的短暂消息延迟
- Master与Slave配对是通过指定相同的brokerName属性来匹配的，Master的brokerId属性必须是0，Slave的brokerId属性必须是大于0的整数
- 一个Master可以对应多个Slave，与同一个Master对应的多个Slave通过制定不同的brokerId属性进行区分

多Master多Slave的异步复制模式的优缺点如下：

- 优点：当Master宕机后，消息实时性不会受影响，因为消费者仍然可以从Slave中消费消息，此过程对应用透明，不需要人工干预。性能与多Master模式几乎一样
- 缺点：在Master宕机和磁盘损坏的情况下，会丢失少量消息

10.3.3、多Master多Slave的同步双写模式

- 每个Master对应一个或多个Slave，有多组Master/Slave
- 写到Master中的消息会通过不写到Slave中，因此称为同步双写，只有Master和Slave都写成功，才会返回消息发送成功的信息

多Master多Slave的同步双写模式的优缺点如下：

- 优点：在Master宕机情况下，消息无延迟，服务可用性与数据可用性都非常高
- 缺点：性能比异步复制模式略低，大约低10%，发送单个消息的响应时间会略高。目前Master宕机后，Slave不能自动切换为Master，后续会支持自动切换功能。

10.4、SpringCloud Stream简介

为了使微服务可以按照统一的方式使用各种消息中间件，SpringCloud Stream（以下简称stream）应运而生。在微服务系统中使用Stream有以下优点

- (1) 是微服务代码与消息中间件解耦。微服务系统不必关心具体使用哪个消息中间件产品，只需按照StreamAPI来收发消息。如果更换了消息中间件，秩序修改以下配置代码，而无须改程序代码
- (2) 学习成本低。开发人员只需了解stream的用法，就能将其与各种消息中间件整合。

Stream的具体实现是通过Binder与特定的消息中间件绑定，针对不同的消息中间件，有相应的Binder实现。Binder绑定的所有通道称为bindings，bindings包括输入通道和输出通道

10.5、在微服务中收发消息

hello-provider微服务充当消息生产者，hello-consumer-service微服务充当消息消费者，生产者通过输出通道发送消息，消费者通过输入通道接收消息

10.5.1、在hello-provider模块及hello-consumer模块的pom.xml文件中，都要加入Stream与RocketMQ的整合依赖

- (1) 在hello-provider模块定义一个Supplier类型的Bean组件

Java.util.function.Supplier属于函数式接口，指定了提供消息的行为，以下Supplier Bean提供的消息为Date对象

```
@Bean
public Supplier<Date> mydate(){
    return ()->new Date();
}
```

在hello-provider的application.yaml中配置函数的定义方法、NameServer的地址和输出通道信息

```
cloud:
  stream:
    function:
      definition: mydate
    rocketmq:
      binder:
        name-server: 127.0.0.1:9876      #NameServer的地址
        group: mygroup
      bindings:
        mydate-out-0:      #输出通道
          destination: date-topic      #输出通道的目的地，此处表示RocketMQ的消息主题
```

- (2) 在hello-consumer模块中定义一个Consumer类型的Bean组件，Java.util.function.Consumer属于函数式接口，指定了消息消费的行为，以下Consumer Bean组件会输出接收到的Date对象

```
@Bean
public Consumer<Date> mydate(){
    return Date date->{
        System.out.println("Received: "+date);
    };
}
```

在hello-consumer的application.yaml中，配置函数的定义方法，NameServer的地址，输入通道信息

```
stream:
  rocketmq:
    binder:
      name-server: 127.0.0.1:9876
    bindings:
      mydate-in-0:      #输入通道
        destination: date-topic      #输入通道的目的地
        content-type: application/json
        group: date-group      #所在的消费组
```

注意：bindings绑定的输入或者输出通道是和方法名对应的

10.2.5、收发Message类型的消息

stream在发送消息时，会自动为消息加上消息头在进行发送。除了固定消息头，还可以在程序中通过SpringMessageAPI来添加自定义的消息头

在hello-provider通过spring message api来添加自定义的消息头，发送的消息为Message<Date>类型

```
@Bean
public Supplier<Message<Date>> msg(){
    return ()->{
        Message<Date> message=MessageBuilder
            .withPayload(new Date())
            .setHeader("headerName:","index",index++)
            .build();
        return message;
    };
}
```

在hello-consumer接收消息类型为Message<Date>类型的消息

```
@Bean
public Consumer<Message<Date>> msg(){
    return Message<Date> message->{
        System.out.println("Received: "+message.getHeaders());
        System.out.println("Received: "+message.getPayload());
    };
}
```

10.6、通过StreamBridge类发送消息

supplier Bean会自动按照固定的时间间隔不停的发送消息。如果希望更具业务逻辑发送消息，可以通过Stream API的StreamBridge类来发送消息，StreamBridge Bean类的特点是能随时随地供生产者调用，向指定的输出通道发送消息

- (1) 在hello-provider中通过@Autowired声明一个StreamBridge Bean组件，该Bean组件由Stream框架自动提供，在sendMessage () 方法中通过StreamBridge Bean组件来发送消息

```
@Autowired
private StreamBridge streamBridge;

@GetMapping("/{username}")
public String sendMessage(@PathVariable String username){
    System.out.println("Sending"+username);
    String jsonData="{\"username\":\""+username+"\"}";
    boolean isSuccess=streamBridge.send("mymsg-out-0",jsonData);
    return isSuccess?"success":"error";
}
```

在hello-provider的application.yaml中，配置mymsg-out-0输出通道

```

stream:
  function:
    definition: mydate;msgs;sendMessage
  rocketmq:
    binder:
      name-server: 127.0.0.1:9876      #NameServer的地址
      group: test-group
    bindings:
      mymsg-out-0:      #输出通道
        destination: test-topic      #输出通道的目的地, 此处表示RocketMQ的消息主题
  nacos:
    discovery:
      server-addr: 127.0.0.1:8818

```

(2) 在hello-consumer模块中声明一个Consumer Bean组件。可以把JSON格式的字串消息转换为符合Java Bean风格的Java对象。定义一个Person内部类，把Consumer的接收类型改为Person

```

@Bean
public Consumer<Person> mymsg(){
    return Person person->{
        System.out.println("Received:"+person);
    };
}

@Data 1 个用法
public static class Person{
    private String username;
    @Override
    public String toString() {
        return this.username;
    }
}

```

10.6.2. 用ChannelInterceptor拦截消息

StreamBridge类通过Spring API中的MessageChannel来发送消息，Spring API对MessageChannel提供了拦截机制，可以通过ChannelInterceptor拦截器在发送和接收消息的过程中增加额外的处理操作

10.7. 发送HTTP请求正文

StreamBridge类会发送HTTP请求路径中的username参数，此外，它也可以发送HTTP请求正文

sendBody () 方法通过StreamBridge类向service-out-0输出通道发送请求正文

```

@PostMapping("@*/submit")
@ResponseStatus(HttpStatus.ACCEPTED)
public void sendBody(@RequestBody String body){
    System.out.println("Sending"+body);
    streamBridge.send( bindingName: "service-out-0",body);
}

```

10.8. 通过Function函数式接口收发消息

Function<T,T>中，第一个类型数据表示接收到的消息；第二个类型数据发送发送的消息。

Function Bean可以作为消息中转站，接受来自Supplier Bean发送的消息，对消息进行处理或过滤，然后把它发送给Consumer Bean

如图10.18中，Supplier Bean通过输出通道origin-out-0发送主题为oldmsg-topic的消息；Function Bean通过输入通道uppercase-in-0接收主题为oldmsg-topic的消息，并对其进行处理。如理，把消息中的文本改为大写

在通过输出通道uppercase-out-0发送主题为newmsg-topic的消息；Consumer Bean通过输入通道target-in-0接收主题为newmsg-topic的消息。



图 10.18 Function Bean 作为消息中转站

10.9. 响应式收发消息

所谓响应式收发消息，是指发送消息和接收消息异步进行，消息以数据流的形式传输，并且消费者可以主动决定何时接收消息，只有当消费者主动订阅消息时，消息才会传送给消费者。响应式编程依赖于Reactive API中的两个核心类

Reactor.core.publisher.Flux类：发送或接收包含0~n个元素的数据流

Reactor.core.publisher.Mono类：发送或接收包含0~1个元素的数据流

10.10. 多输入通道和输出通道

Stream通过与Reactive API整合，来处理多输入通道和输出通道的消息。Reactive API中的Tuple2类用于存放两个通道，如果要存放三个通道，就用Tuple3类，以此类推。。

10.11. 批量消费消息

只要把spring.cloud.stream.bindings.<binding-name>.consumer.batch-mode属性设为true，Stream就支持批量消费消息。如批量接收所有的Person对象，把它们存放在一个List列表中

10.12. 批量生产消息

Stream还支持批量生产消息。例如，把消息放到List列表中，然后一次性发送列表

10.13. 处理错误

当消费消息失败时，Stream提供了以下三种处理方式

- (1) 清除失败消息
- (2) 吧失败消息发送到dql (死信队列)
- (3) 尝试重新消费失败消息

值得注意的是，以上三种处理方式依赖于底层消息中间件的实现，并不是每种消息中间件都支持这三种处理方式。kafka、RabbitMQ和RocketMQ都支持这三种处理方式。此外，这三种处理方式适用于非响应式收发消息。而在响应式收发消息的场景，则可以运用Reactive API本身提供的处理错误的方式。

10.13.1. 清除失败消息

默认情况下，Stream会清除失败消息。在实际生产环境中，不推荐这种方式

10.13.2. 把失败消息发送到dql (死信队列)

默认情况下，对于失败消息，Stream总共尝试3次消费。如果3次消费都遇到异常，在把失败消息发送到DLQ。可以在application.yaml中设置尝试的次数

如果是由由于缓存中存在bug而导致的失败消息，就需要先修改程序代码，等到能正常运行时，通过消息中间件的控制台，把失败消息从DLQ移动到原来的正常队列，再由消费者接收消息

10.13.3. 尝试重新消费失败消息

Stream利用Spring Retry类库中的RetryTemplate类来尝试重新消费失败消息。在application.yaml中，可以设置RetryTemplate类的一些属性，来决定它的重试行为。

还可以创建自定义的RetryTemplate实例，用@StreamRetryTemplate注解标识，并在application.yaml文件中通过Retry-template-name来指定自定义的RetryTemplate Bean

10.14. 消费者分组和分区

- 当多个消费者实例位于同一个消费者组时，对于同一个主题的消息，只能被一个消费者实例消费。Stream会从消费者组中随机找出一个消费者实例来消费一条消息
- 如果希望消费者组中的每个消费者实例固定接收特定的消息，可以对消费者进行分区。每个消费者一个分区索引，如用消息头的id作为分区索引key。Stream会根据key来计算每条消息对应的消费者分区索引。分区索引的计算表达式为key.hashCode()%partitionCount，partitionCount属性标识分区的数目。

10.15. 消息正文的类型转换

消息的消息头有一个contentType属性，表示消息正文的传输类型。Stream在发送消息时，会把消息正文转换为contentType传输类型，然后在网络上传输；接收消息时，会把消息正文再转换为目标类型

针对不同的传输类型，stream提供了相应的MessageConvert。

(1) ApplicationJsonMessageMarshallingConverter: 用于实现POJO类与JSON类型的类型转换。POJO

是指符合Java Bean风格的Java类，对应的传输类型为application/json

(2) ByteArrayMessageConverter: 用于实现byte[]到byte[]的类型转换，对应的传输类型为

Application/octet-stream

(3) ObjectStringMessageConverter: 用于实现任何Java类型与String类型的类型转换，对应的传输类型为text/plain。

(4) JsonUnmarshallingConverter: 用于实现任何Java类型与JSON类型的类型转换，对应的传输类型为application/x-java-object。在设置传输类型时，建议指定具体的Java类型

10.16. 通过Actuator监控Stream

Stream向Actuator暴露了两个端点。

- /actuator/bindings: 用于监控Stream的所有通道信息
- /actuator/health: 用于监控Stream的健康状态

10.17. 小结

分布式微服务再运转的过程中，每个微服务节点之间会频繁交换各种数据。Stream通过整合消息中间件和Java的函数式接口，架起了异步传送数据的通道。生产者通过Supplier函数式接口提供消息，消费者通过Consumer函数式接口消费消息。Spring.cloud.stream.poller.fixed-delay配置属性用于指定Supplier发送消息的间隔时间，默认值为1s。一次Supplier每个1s向输出通道发送一条消息。

Supplier的一个运用范例是股票报价。例如，生产者每隔1s向消费者发送最新的股票报价。

- Stream再传送消息的过程中，会为原始的payload正文消息添加与传送数据有关的消息头。此外，生产者也可以直接发送Message类型的消息，并通过MessageBuild类的setHeader () 方法添加

POJO类： (1) 私有化成员变量； (2) 提供无参构造器； (3) 提供Getter/Setter方法； (4) 无业务逻辑、无框架依赖

Entity/DOT/VO/JavaBean: (1) Entity (实体类)：映射数据库表结构，对应数据库一行数据

(2) DTO (数据传输对象)：跨层/跨服务传递数据 (如UserDTO, Controller<->Service)

(3) VO (视图对象)：像前端页面返回的数据载体 (如UserVO, Controller->前端)

(4) JavaBean: Java官方对POJO的 (官方叫法)，二者完全等价

自定义的消息头

- 生产者还可以通过StreamBridge类的send () 方法向特定的输出通道发送消息。StreamBridge类发送消息的时间由生产者决定
- stream还可以通过Function函数式解耦收发消息，该接口从输入通道接收消息，进行处理后，再向输出通道发送作为处理结果的消息
- stream还可以与响应式编程API整合，使得消费者能够灵活地订阅生产者提供地消息流。Stream与响应式B编程API整合的方式很简单，只需把消息用Mono类或Flux类包装即可。

11、链路追踪组件：skyWalking

1、SkyWalking简介

skyWalking提供了强大的APM（应用性能管理）功能，专门为微服务等基于容器的云原生架构提供监控服务。SkyWalking通过探针收集应用的各项指标，并进行分布式的链路追踪。SkyWalking会感知微服务之间的调用链路的关系，生成相应的统计数据。

skyWalking具有以下特性：

- (1) 支持告警
- (2) 采用探针技术，对业务代码零侵入。所谓零侵入，是指探针不会改变业务代码，也不会改变代码的运行。
- (3) 轻量高效，无需额外的大数据平台
- (4) 提供多种监控手段，支持多语言探针（Java、.Net Core和Node.js）。
- (5) 简介强大的可视化后台管理界面
- (6) 自身采用模块化架构，包括探针（Agent）、UI（User Interface，用户界面）、观测分析平台（OAP）和存储模块

skyWalking的整体架构包括4个模块

- (1) 探针：探针安置在微服务中，负责收集监控数据。
- (2) OAP：接收探针发送的监控数据，利用分析引擎对数据进行整合与运算，把统计数据存储到相应的存储设备中
- (3) UI：调用共OAP接口，再可视化的界面中显示统计数据
- (4) 存储模块：用于存放OAP的统计数据，目前支持的数据库包括H2（内嵌式数据库，是skyWalking的默认数据库）、ES（Elasticsearch）、MySQL、BanyanDB等。



11.2、链路追踪软件

从多个角度对zipkin、CAP和SkyWalking这三款链路追踪软件进行比较

比较角度	软件		
	zipkin	CAT	skyWalking
实现原理	拦截请求	代码埋点（拦截器、过滤器）	探针、字节码增强
探针与OAP之间的协议	HTTP、MQ	HTTP、TCP	rRPC
openTracing	支持	不支持	支持
监控粒度	接口级	代码级	方法级
全局调用统计的协议	HTTP、MQ	HTTP、TCP	gRPC
JVM监控	不支持	支持	支持
告警	不支持	支持	支持
数据存储	ES、MySQL、Cassandra、内存	MySQL、HDFS	ES、H2、MySQL、BanyanDB
可视化UI	支持	支持	支持
聚合报表	少	非常丰富	较丰富
社区支持	国外主流	国内支持	Apache支持
使用案例	京东、阿里巴巴定制后不开源	美团、携程、陆金所	阿里云、华为、小米、当当、百度
APM	不支持	支持	支持
WebFlux	支持	不支持	支持

11.3、安装和运行skyWalking

下载解压skywalking-apm，点击bin目录下的startup.sh，就会启动SkyWalking，实际上启动了两个服务

- oap-service：OAP，其实现类库位于oap-libs子目录下
- Webapp-service：提供UI的webapp应用，位于webapp子目录下

skywalking的webapp-service默认监听的端口为8080，oap-service与探针通信的默认端口为11800，与webapp-service通信的默认端口为12800。确切的说，再oap-service中包含了一个collector-service自服务，它负责收集来自探针的监控数据，它的默认端口为11800。



11.4、在微服务中安置探针

- (1) 下载java-agent压缩包解压，在根目录下有一个skywlking-agent.jar文件，它是探针的类库文件
- (2) 在IDEA中，选择菜单run->Edit Configuration，为启动类添加以下VM Option参数

```
#skywalking-agent.jar的本地地址
-javaagent:D:\publictools\skywalking\apache-skywalking-java-agent-9.0.0\skywalking-age
#skywalking-l显示的服务名
-Dskywalking.agent.service_name=hello-provider-service
#skywalking/collector-service服务的IP及端口
-Dskywalking.collector.backend_service=127.0.0.1:11800
```

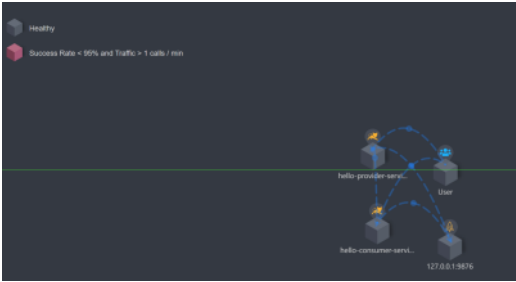
如下图，skywalking控制台会有以下的这些数据。

- Load（数字）：平均每分钟请求数
- Load（折线图）：在一个时间段内每分钟请求数的趋势图
- Success Rate（数字）：请求成功率
- Success Rate（折线图）：在一个时间段内请求成功率的趋势图
- Latency（数字）：平均响应请求的时间，以毫秒为单位
- Latency（折线图）：在一个时间段内响应请求的时间的趋势图，以毫秒为单位
- Apdex（数字）：当前服务的Apdex（应用性能指数）评分
- Apdex（折线图）：在一个时间段内Apdex评分的趋势图

Service Names	Load (calls / min)	Success Rate (%)	Latency (ms)	Apdex
hello-provider-service	1.45	3.23	0.35	0.03
hello-consumer-service	1.16	3.23	1.29	0.03

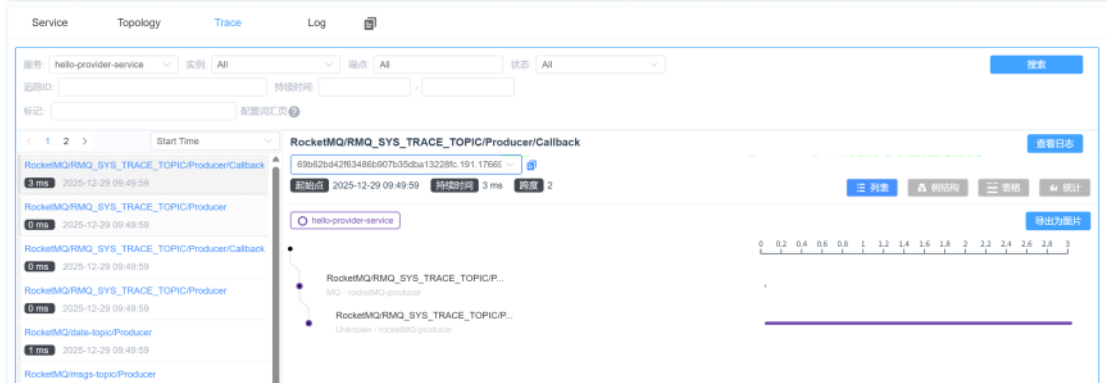
11.4.1、查看拓扑图

选择菜单Topology，页面会显示微服务的拓扑图，展示微服务之间的调用关系。



11.4.2. 追踪链路

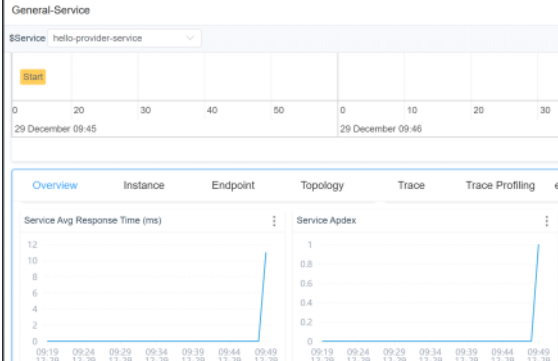
选择菜单Trace，页面中会显示每条调用链链路的详细信息。skywalking为每条调用链路都分配了唯一的追踪ID，并且还会展示链路的持续时间和施一公你的协议等信息



11.4.3. 查看监控数据

skywalking可以从微服务、微服务实例和端点这三种维度查看监控数据

- 点击首页hello-provider-service，就会显示hello-provider-service微服务的监控数据



- 选择菜单Instance，就会从hello-provider-service微服务实例的维度展示监控数据

Service Instances	Load (calls / min)	Success Rate (%)	Latency (ms)	Attributes
76383438128430aadd5624205737a47@172.26.81.248	0.00	0.00	0.00	查看
727cc2416e545c0bc31623d35e902fc@172.26.81.248	1.45	3.23	0.35	查看

Endpoint	Load (calls / min)	Success Rate (%)	Latency (ms)
GET/greet/Tom	1.16	3.23	0.23

- 选择才当Endpoing，就会从端点的维度展示监控数据。这里所谓的端点，是指每个被访问的URL。例如GET: /greet/username就是一个端点

11.4.4. 性能分析

skywalking能够帮助运维人员分析微服务的运行性能，发现问题所在。性能分析不需要在代码中设置埋点，skywalking通过周期性地对业务运行状态保存快照进行性能分析，资源消耗比较小

- (1) 在skywalking的UI选择彩带那个Trace Profiling，就会进入性能分析页面。选择“新建任务”按钮，创建一个监控/enter/username)端点的任务。包括以下选项

- 监控时间：指定从什么时候开始监控，可以选择秒或者自定义时间
- 监控持续时间：指定监控的时间长度
- 起始监控时间：多长时间采集样本
- 监控间隔：采集样本的间隔，即执行快照的间隔
- 最大采样数：最多采集多少次样本

端点名称
GET/enter/Tom

监控时间
☒ monitor now ☐ set start time 2025-12-29 09:53:10

监控持续时间
☒ 5 min ☐ 10 min ☐ 15 min

起始监控时间 (ms)
0

监控间隔
☒ 10 ms ☐ 20 ms ☐ 50 ms ☐ 100 ms

最大采样数
1

新建任务

11.5. 采集日志

skywalking还能采集应用程序输出的日志，以下是为hello-consumer微服务采集日志的步骤

- (1) 在pom.xml中引入日志工具依赖

```
<dependency>
  <groupId>org.apache.skywalking</groupId>
  <artifactId>apm-toolkit-logback-1.x</artifactId>
  <version>8.11.0</version>
</dependency>
```

- (2) 在hello-consumer中增加logback日志工具地logback-spring.xml文件。<pattern>元素指定日志输出格式；tid表示链路追踪ID

<?xml version="1.0" encoding="UTF-8"?>

- (3) 在skywalking的config的agent.conf文件中配置collector-service的服务的地址和端口
- (4) 可以在skywalking的Log菜单中查看日志的详细信息

Overview	Inance	Endpoint	Topology	Trace	Trace Profiling	eBPF Profiling	Log	
服务:				hello-consumer-service				
实例:				8e2e4407012443b79583090c75c696b9@172.26.81.248				
端点:				GET/enter/Tom				
跨度类型:				Entry				
组件:				Tomcat				
Peer:				No Peer				
错误:				false				
标记:								
url:				http://localhost:8082/enter/Tom				
http.method:				GET				
http.status_code:				200				

微服务的调用链路中的URL (如/greet/{username})或/enter/{username}) 会作为skywalking的监控端点。对于普通方法, 需要通过skywalking的@Trace注解将其标识为监控端点 (也称为埋点)。如在hello-consumer微服务中将一个普通方法add () 设为监控端点

```
<dependency>
  <groupId>org.apache.skywalking</groupId>
  <artifactId>apm-toolkit-trace</artifactId>
  <version>8.11.0</version>
</dependency>
```

还可以为add () 方法加上@Trace标签, 用于追踪方法的参数和返回值。在add () 方法中, 还可以通过TraceContext类向追踪上下文容器存取key/value数据, 以及通过traceld () 获得当前链路的TraceID

(3) 访问localhost/sum/5/6

11



The screenshot displays a REST client interface. The top section shows a list of GET requests to the endpoint `/sum/5/6`. The first request is selected, and its details are shown in the bottom section. The details include the URL `http://localhost:8080/sum/5/6`, the method `GET`, and the headers `Host: localhost:8080`. The response status is `200 OK`.

11.7、忽略端点

11.6介绍了把普通方法设为可以被skywalking追踪的端点的步骤，而在有些场景中，需要让skywalkgin忽略追踪一些端点，因为没有必要采集这些端点的监控数据。指定忽略hello-consumer微服务的一些端点的配置步骤如下

- (1) 把Optional-plugins目录下的apm-trace-ignore-plugin-8.11.0.jar文件负责到plugins目录下
- (2) 在confi目录下新增一休息apm-trace-ignore-plugin.conf文件，指定需要忽略的路径。内容如下，指定skywalking忽略的路径为/actuator/health/**和/list，多个路径之间以英文逗号隔开

Trace.ignore_path=\${SW_AGENT_TRACE_IGNORE_PATH:/actuator/health/**,/list}

在路径中可以加入通配符。例如：

- /mypath/? 标识匹配单个字符
- /mypath/*表示匹配多个字符
- /mypath/**表示匹配多个字符并且支持多级目录

除了在apm-trace-ignore-plugin.conf文件中指定需要忽略的端点，还可以通过设置skywalking.trace.ignore_path系统属性来设定。例如，在IDEA中选择菜单Run->Edit Configurations，对HelloConsumerApplication的自动配置增加以下vm option参数

11.8、告警

在skywalking安装目录的config/alarm-setting.yml文件中，已经设置了默认的告警规则。

在hello-consumer编写满足告警规则的方法

```
@GetMapping("@*/"/timeout")
public String timeout(){
    try {
        Thread.sleep( millis: 2000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    return "timeout";
}
```

创建处理告警的webhook，创建一个notify () 方法，它映射的URL为/notify，充当处理告警的webhook。产生告警时，skywalking会调用notify () 方法，该方法的HTTP请求方式为POST，notify () 方法的参数用@RequestBody注解标识。

```
@PostMapping("@*/"/notify")
public void notify(@RequestBody List<AlarmMessage> alarmMessages){
    alarmMessages.forEach( AlarmMessage value->{
        System.out.println(value);
    });
}
```

修改skywalking的告警规则配置文件config/alarm-setting.yml，将webhook的地址修改为以下/notify的URL地址

```
webhooks:
- http://127.0.0.1:8082/notify
```

访问http://localhost:8082/timeout，skywalking的UI会显示告警信息



skywalking会通过告警配置文件调用/notify打印出告警信息

```
AlarmMessage{scopedId=null,name=98dd87932221489d8738d528e14c3cdc@172.26.81.248 of hello-consumer-service,id8=a6Vsb68tY29uc3VtZX
AlarmMessage{scopedId=null,name=User in User to GET:/timeout in hello-consumer-service,id8=VXNlcg==.0_VXNlcg==,id1=a6Vsb68tY29u
```

图11.22展示了产生告警的流程



图 11.22 产生告警的流程

在实际应用中，可以在webhook的接口的时间中对短信、邮件等平台，确保当告警出现时，能迅速把告警消息发送给相应的处理人员，提高故障处理的速度

11.9、整合elasticsearch数据库

默认情况下，skywalking把监控数据存储在H2数据库中，H2是一个内嵌在skywalking中的数据库，以内存作为存储介质。因此，当skywalking重启后，原先的监控数据就丢失了。如果希望永久地存储监控数据，可以使用elasticsearch或MySQL数据库

elasticsearch用Java语言开发，是一种流行地企业级搜索引擎数据库。skywalking整合elasticsearch数据库地步骤如下：

- (1) 下载elasticsearch的zip包解压。修改config/elasticsearch.yaml配置文件

=====配置节点和端口等信息=====

```
Cluster.name: my-application #集群名字
Node.name: node-1 #节点名字
Network.host: 0.0.0.0
Http.port: 9200 #监听的端口
Discovery.seed_hosts: ["127.0.0.1"] #主节点的地址
Cluster.initial_master_nodes: ["node-1"] #主节点的名字与node.name一致
# ===== 核心：关闭ES.x安全认证 + HTTPS =====
xpack.security.enabled: false
xpack.security.http.ssl.enabled: false
xpack.security.transport.ssl.enabled: false
访问http://localhost:9200
```

```
localhost:9200
{
  "name": "node-1",
  "cluster_name": "my-application",
  "cluster_uuid": "WZ6B8y8S2edXq2vKJGJnQ",
  "version": {
    "number": "8.3.0",
    "build_type": "zip",
    "build_hash": "5b8b98164fcd1ba1d88751646b4b461b4cc",
    "build_date": "2022-06-23T22:48:49.607492124Z",
    "build_snapshot": false,
    "lucene_version": "9.2.0",
    "minimum_wire_compatibility_version": "7.17.0",
    "minimum_index_compatibility_version": "7.0.0"
  },
  "tagline": "You Know, for Search"
}
```

修改skywalking目录下的config/application.yml文件，指定把监控数据存储在Elasticsearch中。修改以下4项

```
Selector: ${SW_STORAGE:elasticsearch}
Namespace: ${SW_NAMESPACE:my-application}
User: ${SW_ES_USER:elastic}
Password: ${SW_ES_PASSWORD:ysjlf}
```

11.10、整合MySQL数据库

- (1) 安装mysql
- (2) 下载mysql驱动程序类库mysql-connector-java-8.0.11.jar，把它复制到skywalking安装目录的oap-lib子目录下
- (3) 修改skywalking目录下的config/application.yml文件，指定用mysql存储监控数据，并配置连接mysql的属性。

11.12、通过Nacos简历SkyWalking集群

skywalking包括oap-service和webapp-service两个服务，都可以对这两个服务分别建立集群。

对oap-service建立集群：如图11.25，有两个oap-service节点，oap-service1节点监听的端口为11801和12801，oap-service2节点监听的端口为11802和12802.这两个节点都注册到同一个nacos服务器，并且访问同一个elasticsearch数据库。这样就能保证两个节点上监控数据的一致性

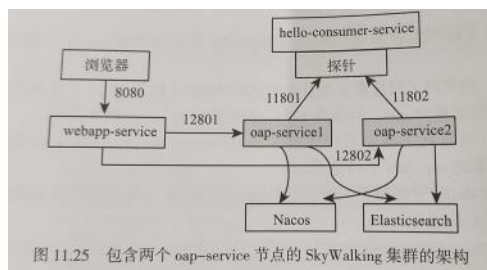


图 11.25 包含两个 oap-service 节点的 SkyWalking 集群的架构

为webapp-service服务通过nginx建立集群，在nginx目录下的conf/nginx.conf为两个webapp-service节点配置节点代理



图 11.26 webapp-service 服务的集群

为两个 oap-service 通过 nginx 遍历集群的具体步骤如下：

- (1) 配置两个 skywalking 节点的 config/application.yml，使它们都连接到同一个 Elasticsearch 数据库
- (2) 修改两个 skywalking 节点的 config/application.yml 文件，将它们都注册到 Nacos。（是直接写在 skywalking 节点的 config/application.yml 文件配置 Nacos）
- (3) 修改两个 skywalking 节点的 config/application.yml 设置它们监听的端口
- (4) 修改 webapp/webapp.yml 文件，加入两个 oap-service 节点的地址，以英文都好隔开
- (5) 修改 hello-consumer 模块的启动配置，在 VM Option 参数中加入两个 oap-service 节点的 collector-service 服务的地址。

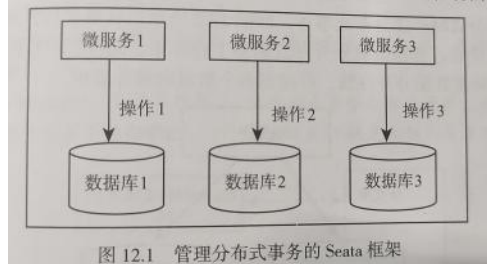
11.12、小结

- 当用户向微服务系统发出一个请求时，该请求会由多个微服务共同协作来生成响应结果，如果有一个环节出了故障，就会导致响应失败。skywalking 通过探针追踪微服务的调用链路，监控每个请求的响应时间、响应状态，以及有哪些端点参与对请求的响应等。如果满足告警规则，系统还会发出告警，帮助运维人员即使发现链路中的隐患
- skywalking 支持把监控数据永久保存到 Elasticsearch 或 MySQL 等数据库中，还可以通过 Nacos 建立服务的集群，并确保集群中的 oap-service 节点之间监控数据的一致性
- skywalking 包括以下两个服务
 - oap-service 服务：收集并分析由探针采集的监控数据，其 collector-service 自服务监听探针的默认端口为 11800。该服务向 webapp-service 提供监控数据的默认端口为 12800
 - webapp-service 服务：提供展示监控数据的 UI，默认端口为 8080

12、分布式事务管理框架：Seata

seata 是由阿里巴巴公司开发的一款分布式事务管理框架。seata 为分布式微服务系统提供了同一管理分布式事务的平台。微服务本身无需考虑多个数据库之间进行协调的通信细节，秩序声明事务的开始、提交和撤销，seata 就会协调多个数据库共同完成事务中的所有操作，事务执行成功就提交所有的操作，否则就撤销所有的操作。

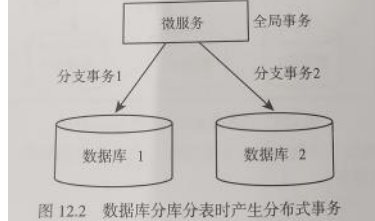
如图 12.1 中三个微服务分别访问三个数据库，完成各自的操作，而这些操作属于由 seata 框架管理的同一个分布式事务



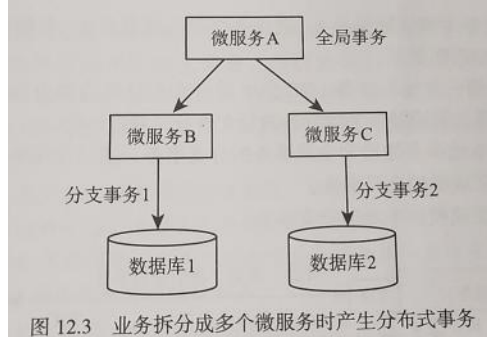
12.1、分布式事务概述

数据库事务必须具备 ACID 特性。如果事务的操作会跨越多个数据库，则这样的事务称为分布式事务。才外，如果由多个进程操作同一个数据库，共同完成一个事务，这样的事务事务页称为分布式事务

(1) 数据库分库分表时产生分布式事务



(2) 一万五拆分成多个微服务时产生分布式事务



12.2、Seata 简介

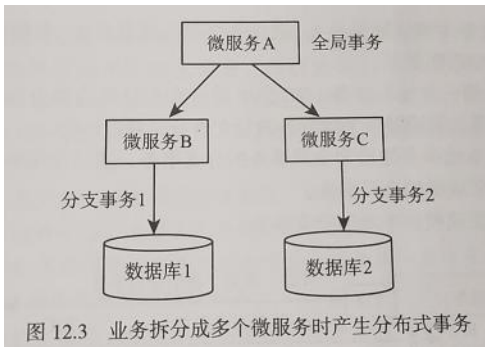
seata 是一款开源的分布式事务框架，用于在微服务架构中提供高性能和简单易用的分布式事务管理服务。在 Seata 框架中，一共有以下三个角色

TC（事务协调者）：管理全局和分支事务的状态，驱动分支事务的提交和撤销，从而提交或撤销全局事务

TM（事务管理器）：声明全局事务的边界，包括开启全局事务、提交或撤销全局事务

RM（资源管理器）：管理分支事务处理的资源，与 TC 交互，向 TC 注册分支事务并报告分支事务的状态，TC 会驱动 RM 提交或撤销分支事务

TC 时单独部署的服务器，称为 seata Server，TM 和 RM 是嵌入到微服务中的客户端组件。如 12.5 中就共同完成了一个分布式的分布式全局事务。



12.3. Seata的事务模式

按照管理分布式事务的不同方式，Seata有以下4种事务方式。

- AT模式：采用两阶段提交模型，依赖底层数据库对ACID特性的支持
- TCC模式：采用两阶段提交模型，不依赖底层数据库对ACID特性的支持，把自定义的分布式事务纳入到全局事务的管理中
- Saga模式：提供长事务解决方案，为异构系统提供事务的统一处理模型。
- XA模式：利用事务资源（数据库、应用服务器、消息队列等）对XA协议的支持，以XA协议的机制来管理分支事务

以上的两阶段提交模型是指Seata把分支事务的生命周期分为以下两个阶段。

- 第一阶段：在本地数据库执行分支事务，并保证执行结果满足两个条件：可持久化和可撤销。本阶段向TC汇报分支事务的执行状态
- 第二阶段：根据执行阶段的结果，TM向TC发出全局提交或撤销请求，TC再命令RM提交或撤销分支事务

12.3.1. AT模式

在AT模式下，管理分支事务包括以下两个阶段：

- 第一阶段：将业务数据和回滚日志记录在同一个本地事务种提交，接着释放本地锁和数据库连接
- 第二阶段：异步提交分支事务，具有很快的响应速度。如果要撤销分支事务，就会通过第一阶段的回滚日志记录进行反向补偿

AT模式的工作机制包括：写隔离和读隔离

- (1) 写隔离：当两个全局事务同时更新一张数据库表中的相同记录时，为了避免对共享数据的竞争，seata采用全局锁进行写隔离，具体遵循以下原则
 - 在提交本地事务前，必须先拿到全局锁
 - 拿不到全局锁，就不能提交本地事务
 - 只能在一定范围内尝试获得全局锁，如果超出范围，将放弃获得全局锁并撤销本地事务，同时释放本地锁

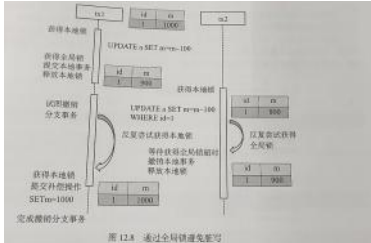
本地锁时本地数据库系统提供的，该锁只能隔离多个操纵本地数据库的事务。而全局锁是由seata产生的，用于隔离多个并发的分布式全局事务

如图12.7，tx1和tx2提交各自全局事务的流程如下：

- (1) tx1开启本地事务，获得本地锁，执行更新操作m=m-100
- (2) tx1的本地事务提交前，鲜活的该记录的全局锁，本地事务提交后就释放本地锁
- (3) tx2开启本地事，获得本地锁，执行更新操作m=m-100
- (4) tx2的本地事务提交前，尝试获取该记录的全局锁。此时tx1持有全局锁，tx2需要等待并尝试重新获取全局锁
- (5) tx1执行执行第二阶段的提交全局事务，释放全局锁。Tx2获得全局锁后提交本地事务。



如果tx1在第二阶段撤销分支事务，则tx1需要重新获得该数据的本地锁，进行反向补偿的更新操作，显示分支事务的回滚。在整个过程中，全局锁在tx1结束后一致被tx1持有，所以不会发生tx2脏写的问题。



(2) 读隔离

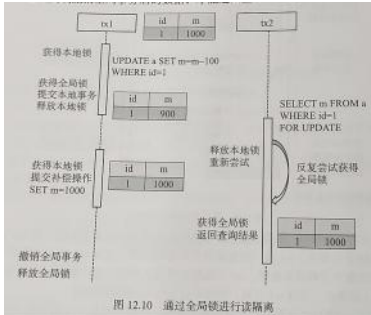
本地数据库系统提供4种事务隔离级别，由高到低依次如下：

- serializable：串行化
- Repeatable Read：可重复读
- Read Committed：读已提交
- Read Uncommitted：读未提交

在数据库本地事务的隔离级别为Read Committed或以上的基础上，Seata在AT模式下默认的全局事务的隔离几倍是Read Uncommitted，如果要求全局事务的隔离级别为Read Committed，目前Seata的解决

方案是对SELECT或UPDATE语句进行代理，要求必须先获得全局锁，才能得到查询结果。

如图12.10，tx2执行SELECT或UPDATE语句时需要先申请全局锁，如果全局锁被tx1持有，则tx2释放本地锁并重试。在这个过程中，tx2的查询操作处于阻塞状态，直到获得了全局锁，读到tx1已提交或撤销全局事务后的数据，才能返回查询结果



12.3.2. TCC模式

全局事务由若干分支事务组成，分支事务采用两阶段提交模型。概括起来，两个阶段具有以下行为。

第一阶段prepare行为。第二阶段commit或rollback行为。

更具两阶段提交模型的具体实现方式，seata提供了AT模式和TCC模式。AT模式依赖本地关系数据库对事务的ACID特性的支持。

- 第一阶段的prepare行为：在本地事务中，提交业务数据更新以及相应的undo_log日志记录。
- 第二阶段的commit行为：立即成功结束，返回事务执行成功的响应结果，并自动异步批量清理undo_log日志记录。
- 第二阶段的rollback行为：通过undo_log日志记录，自动生成补偿操作，完成数据回滚

TCC模式的特定是不依赖于底层数据库对本地事务的支持，而是把自定义的分支事务纳入到全局事务的统一管理中。

- 第一阶段的prepare行为：调用自定义的prepare逻辑

- 第二阶段的commit行为：调用自定义的commit逻辑
- 第三阶段的rollback行为：调用自定义的rollback逻辑

12.3.3、Saga模式

在Saga模式中，当出现某个参与者提交事务失败，则补偿前面的参与者已经成功提交的事务。第一阶段的正向事务和第二阶段补偿事务全部通过开发相关业务代码来实现

saga模式的适用场景包括以下两种：

- 业务流程长且多
- 参与者包含其他公司开发的服务或遗留系统服务，无法提供tcc模式所需的接口

saga模式的优势如下：

- 第一阶段提交本地事务，无须全局锁，性能高
- 采用事件驱动架构，参与者可异步执行，具有高吞吐性能
- 易于按补偿服务

saga模式的缺点是不能保证事务的隔离性，在极端情况下可能由于脏写而无法完成回滚操作。

12.3.4、XA模式

XA协议是由x/open组织定义的分布式事务处理规范。XA模型利用事务资源（如数据库、应该用服务器、消息队列等）对XA协议的支持，以XA协议的机制来管理分支事务。

分支事务的第一阶段具有以下特点

- 可回滚：业务SQL操作在XA分支事务中进行，依赖事务资源对XA协议的支持来保证可回滚。
- 可持久化：XA分支事务完成后，执行XA prepare行为，同样，依赖事务资源对XA协议的支持来保证可持久化。

分支事务的第二阶段具有以下特点

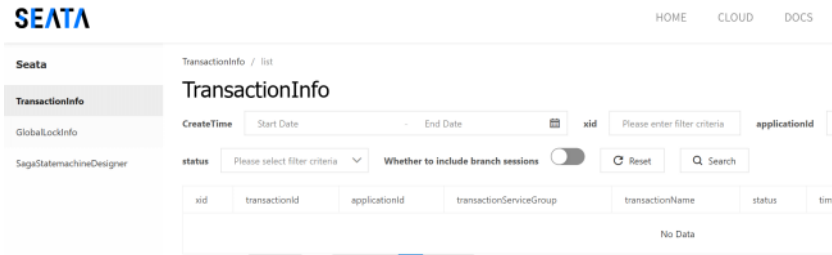
- 分支提交：执行XA分支事务的commit行为
- 分支回滚：执行XA分支事务的rollback行为

seata的AT、TCC、Saga模式都是补偿型的。补偿型事务处理机制依赖于本地事务，但事务资源本身对分布式事务的无感知的，其无法做到真正的全局一致性。与补偿型事务吃力方式不同，XA协议要求事务资源本身提供对XA协议的支持。这样事务资源就能感知并参与分布式事务处理过程，所以事务资源（如数据库）可以保证有效隔离任何事务对数据的访问，从而满足数据的一致性。

12.4、安装和运行Seata Server

在seata框架中，Seata Server承担着TC的角色，负责事务的总协调。Seata Server会记录全局事务和分支事务的信息。这些事务信息由三种存储模式，分别是file模式、db模式和redis模式。

下载seata-server-1.7.0.zip文件并解压。运行/bin/seata-server.bat文件，就会启动seata server，默认配置下，seata server的事务监听端口为8091（application.yml文件中的server.port属性指定内嵌的Tomcat监听的端口，而监听事务的端口为server.port+1000）。存储事务信息的文件为bin/sessionStore/root.data



12.5、创建AT模式下的范例

本节将创建一个模拟电子商务中商品交易业务的storeapp范例。storeapp范例包括以下4个微服务

- business-service微服务：处理全局的商品交易事务，会先后调用共stock-service微服务和order-service微服务
- stock-service微服务：处理减库存的分支事务
- order-service微服务：处理生成订单的分支事务，并且会调用account-service微服务
- account-service微服务：处理修改账户余额的分支事务

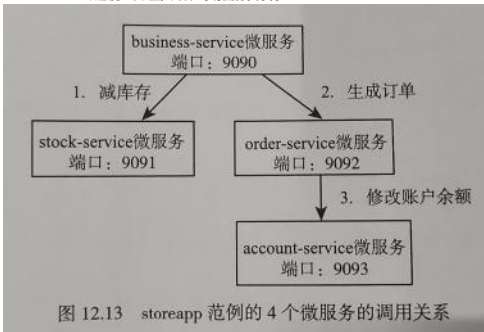


图 12.13 storeapp 范例的 4 个微服务的调用关系

如图12.14所示，商品交易全局事务包括以下三个分支事务。减库存和生成订单这两个分支事务直接嵌套在商品交易全局事务中，而修改账户余额分支事务嵌套在生成订单分支事务中。

- (1) 减库存分支事务
- (2) 生成订单分支事务
- (3) 修改账户余额分支事务

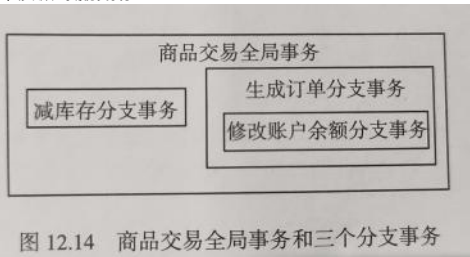


图 12.14 商品交易全局事务和三个分支事务

12.5.2、处理分布式事务的微服务的架构

stock-service、order-service、account-service微服务都会通过访问数据库来完成具体的分支事务。business-service和order-service微服务还会通过OpenFeign调用其他的微服务。

如order-service微服务，其软件架构图如图12.15

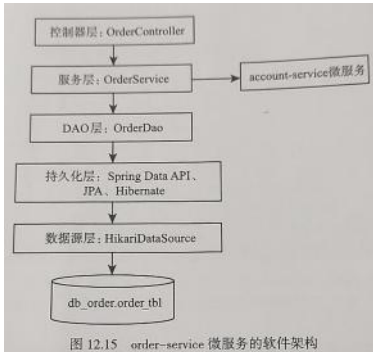


图 12.15 order-service 微服务的软件架构

数据库连接池选择：HikariCP vs Druid:

HikariCP:

轻量级，高性能的数据库连接池。

占用较少的内存和CPU资源

支持快速的连接获取和释放

Druid:

提供了丰富的监控和统计功能。可以实时查看数据库连接的使用情况

提供了防止恶意攻击和SQL注入的功能

提供了连接泄露检测和慢查询日志等功能

business-service微服务本身不会访问数据库，只会通过OpenFeign调用共stock-service微服务和order-service微服务，其软件架构图如图12.16所示



12.5.3. 创建order-service微服务

- (1) 在pom.xml中引入seata和mysql依赖
- (2) 在application.properties文件配置数据源和seata
- (3) 创建DataSourceConfig类：order-service微服务通过Hikari数据源连接数据库。DataSourceConfig类用于声明HikariDataSource Bean。

```

@Configuration
public class DataSourceConfig {
    // ~
    protected static <T> T createDataSource(DataSourceProperties properties, 1个用法
        Class<? extends DataSource> type) {
        return (T) properties.initializeDataSourceBuilder().type(type).build();
    }
    // ~
    @Bean
    @ConfigurationProperties(prefix = "spring.datasource.hikari")
    public HikariDataSource hikariDataSource(DataSourceProperties properties) {
        HikariDataSource dataSource = createDataSource(properties,
            HikariDataSource.class);
        if (StringUtils.hasText(properties.getName())) {
            dataSource.setPoolName(properties.getName());
        }
        return dataSource;
    }
    // ~
    @Primary
    @Bean("dataSource")
    public DataSource dataSource(HikariDataSource hikariDataSource) { return new DataSourceProxy(hikariDataSource); }
}
  
```

- (4) 创建order实体类：

@Entity: 标记当前Java类是JPA的实体类，表示这个类会一对一映射到数据库的某张表，是JPA识别实体类的【开关注解】
 @Table: 指定当前实体类映射的数据库表名，解决【Java类名!=数据库表名】的问题
 @DynamicUpdate: 动态更新，执行update更新操作时，只生成有值字段的SQL语句，忽略null的字段
 @DynamicInsert: 动态插入，执行insert新增操作时，自动忽略值为null的字段，仅插入有值字段
 @Id: 标记当前属性时数据库表的主键字段，是JPA识别主键的唯一注解，必须加在主键属性上
 @GeneratedValue (strategy=GenerationType.IDENTITY: 指定主键的自动生成规则，无需手动设置主键ID，彻底解决主键重复问题
 GenerationType是主键生成策略枚举，IDENTITY表示主键自增，依赖数据库的【自增主键】特性 (mysql的auto_increment)，主键由数据库自动生成

```

@Entity
@Table(name = "order_tbl")
@DynamicUpdate
@DynamicInsert
@Data
public class Order {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(name = "user_id")
    private String userId;
    @Column(name = "product_code")
    private String productCode;
    @Column(name = "money")
    private BigDecimal money;
    @Column(name = "count")
    private Integer count;
    public Order() { }
    public Order(String userId, String productCode, BigDecimal money, Integer count) {
        this.userId = userId;
        this.productCode = productCode;
        this.money = money;
        this.count = count;
    }
}
  
```

- (5) 创建访问数据库的orderdao接口：orderdao接口继承了JpaRepository，让orderdao拥有了【开箱即用的全套数据库操作能力】，无需编写任何SQL/实现类，就能完成95%的日常数据库操作，是Spring Data JPA的核心优势。Spring框架会自动为orderdao提供具体的实现

```

public interface OrderDao extends JpaRepository<Order, Long> {
}
  
```

- (6) 创建AccountFeignClient接口：该接口通过OpenFeign访问account-service微服务

```

@FeignClient(value = "account-service") 2个用法
public interface AccountFeignClient {
    // ~
    @GetMapping("/debit")
    Boolean debit(@RequestParam("userId") String userId, @RequestParam("money") BigDecimal money);
}
  
```

- (7) 创建orderservice类

orderservice类通过orderdao接口向数据库插入新订单，同时通过AccountFeign接口访问account-service微服务。 (消费者通过Feign接口来访问提供服务的提供者，提供者者在controller层实现接收这个接口的访问)

```

@Transactional //生成订单分支事务 1个用法
public void create(String userId, String productCode, Integer count) {

    BigDecimal orderMoney = new BigDecimal(count).multiply(new BigDecimal( val 5));

    Order order = new Order();
    order.setUserId(userId);
    order.setProductCode(productCode);
    order.setCount(count);
    order.setMoney(orderMoney);

    orderDAO.save(order);
    //修改账户余额分支事务
    accountFeignClient.debit(userId, orderMoney);

}

public List<Order> findAll() { return orderDAO.findAll(); }

```

(8) orderController类通过调用Orderservice服务类的方法生成订单。ordercontroller类为orderservice微服务提供基于URL形式的对外接口“/create”。business-service微服务会通过该接口访问order-service微服务。

```

@GetMapping(value="/create")
public Boolean create(String userId, String productCode, Integer count) {
    orderService.create(userId, productCode, count);
    //orderService.create("USER_0001", "PRODUCT_0001", 1);
    return true;
}

@GetMapping(value="/list")
public List<Order> findAll() { return orderService.findAll(); }
}

```

12.5.4. 声明全局事务和分支事务

在business-service微服务中，businessService类的purchase () 方法通过@GlobalTransactional注解声明全局事务

```

@GlobalTransactional //全局商品交易事务 2个用法
public void purchase(String userId, String productCode, int orderCount) {
    //减库存分支事务
    stockFeignClient.deduct(productCode, orderCount);
    //生成订单分支事务
    orderFeignClient.create(userId, productCode, orderCount);
}

```

stock-service微服务的stockService类、accountn-service微服务的accountnService类、order-service的orderservice类都通过@Transactional注解声明分支事务

```

@Transactional //减库存分支事务 1个用法
public void deduct(String productCode, int count) {
    Stock stock = stockDAO.findByProductCode(productCode);
    stock.setCount(stock.getCount() - count);

    stockDAO.save(stock);
}

```

```

@Transactional //生成订单分支事务 1个用法
public void create(String userId, String productCode, Integer count) {
    BigDecimal orderMoney = new BigDecimal(count).multiply(new BigDecimal( val 5));

    Order order = new Order();
    order.setUserId(userId);
    order.setProductCode(productCode);
    order.setCount(count);
    order.setMoney(orderMoney);

    orderDAO.save(order);
    //修改账户余额分支事务
    accountFeignClient.debit(userId, orderMoney);

}

```

```

@Transactional(rollbackFor = Exception.class) //修改账户余额分支
public void debit(String userId, BigDecimal num) {
    Account account = accountDAO.find(userId);
    account.setMoney(account.getMoney().subtract(num));
    accountDAO.save(account);

    if (ERROR_USER_ID.equals(userId)) {
        throw new RuntimeException("account branch exception");
    }
}

```

12.6. 搭建与Nacos整合的Seata Server集群

为了保证seata server的高可用性，可以建立与nacos整合的seata server集群。Seata server记录事务信息的三种存储模式：file、db和redis。在集群环境中，只能db和redis存储模式，才能保证多个seata server节点共享

存储在数据库或redis中的事务信息。

图12.17展示了seata server集群的架构。两个seata server节点被注册到nacos服务器，并且都以同一个mysql数据库作为事务信息的存储源。微服务首先从nacos服务器中得到seata server节点的列表，然后访问seata server集群中的节点

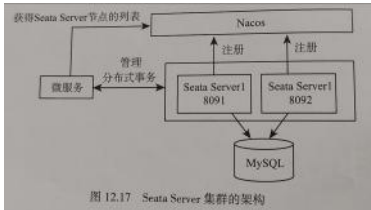


图 12.17 Seata Server 集群的架构

搭建seata server集群的步骤如下：

- (1) 在mysql中创建供seata server访问的seata数据库和表
- (2) 在nacos配置中心配置seata server
- (3) 在seata server的配置文件中配置nacos
- (4) 在微服务中配置seata

12.6.1. 在mysql中创建seata数据库和表

通过如下命令导入seata事务有关的表
source D:/publictools/seata/incubator-seata-1.7.0/incubator-seata-1.7.0/script/server/db/mysql.sql;

```
mysql> show tables;
+-----+
| Tables_in_seata |
+-----+
| branch_table    |
| distributed_lock |
| global_table     |
| lock_table      |
+-----+
4 rows in set (0.00 sec)
```

12.6.2、在nacos配置中心配置seata server

在nacos管理平台创建新的配置管理，添加需要配置的seata server的mysql存储源

```
store.mode=db
store.db.datasource=druid
store.db.dbtype=mysql
store.db.driverClassName=com.mysql.cj.jdbc.Driver
store.db.url=jdbc:mysql://127.0.0.1:3306/seata?useUnicode=true&rewriteBatchedStatements=true
store.db.user=root
store.db.password=ysj1f
store.db.minConn=5
store.db.maxConn=30
store.db.globalTable=global_table
store.db.branchTable=branch_table
store.db.distributedLockTable=distributed_lock
store.db.queryLimit=100
store.db.lockTable=lock_table
store.db.maxWait=5000
```

12.6.3、在seata server的配置文件中配置nacos

在seata目录conf/application.yml中配置nacos，将seata server注册到nacos服务器，并从中获取配置单元seataServer.properties的配置内容

```
console:
  user:
    username: seata
    password: seata
seata:
  config:
    # support: nacos, consul, apollo, zk, etcd3
    type: nacos
    nacos:
      server-addr: 127.0.0.1:8810
      namespace:
      group: SEATA_GROUP
      username: nacos
      password: nacos
      data-id: seataServer.properties #配置单元seata ID，从nacos获取mysql存储源配置值
  registry:
    # support: nacos, eureka, redis, zk, consul, etcd3, sofa
    type: nacos
    nacos:
      application: seata-server
      server-addr: 127.0.0.1:8810
      group: SEATA_GROUP
      namespace:
      cluster: default
      username: nacos
      password: nacos
```

12.6.4、在微服务中配置Seata，在4个微服务的application.properties文件中配置seata，如下：

```
seata.registry.type=nacos
seata.registry.nacos.cluster=default
seata.registry.nacos.server-addr=127.0.0.1:8810
seata.registry.nacos.group=SEATA_GROUP
seata.registry.nacos.application=seata-server
```

12.6.5、访问seata server集群

启动nacos、seata和4个微服务后，在nacos出现如下的服务列表

服务列表

public | mydev

服务名称 请输入服务名称 分组名称 请输入分组名称 隐藏空服务

服务名	分组名称	集群数目	实例数	健康实例数
order-service	DEFAULT_GROUP	1	1	1
account-service	DEFAULT_GROUP	1	1	1
seata-server	SEATA_GROUP	1	2	2
stock-service	DEFAULT_GROUP	1	1	1
business-service	DEFAULT_GROUP	1	1	1

12.7、事务组与seata server集群的映射

如果是一个大型分布式微服务系统，有多个事务组和seata server汲取你，这是就很有必要设定事务组与seata server集群的映射关系，即指定各个seata server集群负责管理的事务组，有两种方式

(1) 在微服务的application.properties文件中，配置事务组的名字，以及指定事务组对应的seata server集群的名字。

```
seata.tx-service-group=my_test_tx_group #指定事务组的名字
seata.service.vgroup-mapping.my_test_tx_group=default
seata.registry.type=nacos
seata.registry.nacos.cluster=default #指定事务组对应的seata server集群的名字
```

在seata server安装目录的application.yml文件中，指定seata server所在的集群的名字

```
seata.id: seata-server-1234567890 #指定ID
registry:
  # support: nacos, eureka, redis, zk, consul, etcd3, sofa
  type: nacos
  nacos:
    application: seata-server
    server-addr: 127.0.0.1:8810
    group: SEATA_GROUP
    namespace:
    cluster: default #seata server所在集群的名字
```

(2) 可以在nacos配置中心进行配置，微服务再从nacos中读取该配置

12.8、小结

关系型数据库会保证本地事务的ACID特性，并且当并发事务访问共享数据时，会提供4中隔离级别：序列化、可重复读、读已提交、读未提交，当分布式的微服务访问分布式数据库时，就会涉及处理分布式事务。seata通过Seata Seata来对各个分支事务进行协调，确保全局事务中的分支事务要么全部提交，要么全部撤销。Seata会通过全局锁来对并发的多个分布式事务进行隔离。

13、分库分表中间件：shardingSphere

在第12章中，为了扩展数据库存储容量并提高数据库的运行性能，可以把表分开存放到不同的数据库中。例如，把账户表、订单表、库存表分别放到不同的数据库中，如图13.1。



图 13.1 拆分数据库

把订单表拆分成订单表1和订单表2，发呢存放到两个数据库中，拆分订单表的过程称为分库分表。如图13.2

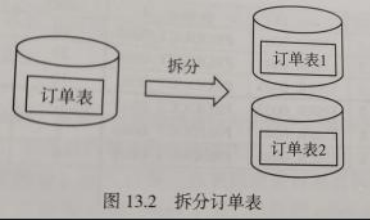


图 13.2 拆分订单表

分库分表会带来新的问题，对表进行插入、更新或删除等操作会很麻烦，例如，应用程序必须增加额外的逻辑，判断把新数据插入到哪张分表中。可以运用ShardingSphere中间件来封装分库分表的细节，为应用程序提供完整的逻辑表。当微服务通过ShardingSphere中间件来访问订单表order_1和订单表order_2。ShardingSphere中间件向微服务提供了虚拟的order逻辑表，它包含了完整的订单数据。微服务只需访问一张order逻辑表即可，无须考虑分库分表的细节。



图 13.3 微服务通过 ShardingSphere 中间件来访问分库分表的订单数据

13.1、分库分表概述

对表进行拆分时，通常把拆分后的表放到不同的数据库中，因此分表往往会导致分库。分表有以下两种方式

- 水平拆分：按照记录拆分表
- 垂直拆分：按照字段拆分表

13.1.1、水平拆分的优点如下：

- (1) 限制单库单表的数据库量，有助于提高性能
- (2) 拆分表的逻辑相同，便于应用程序按照特定的路由规则进行访问
- (3) 提高了整个数据库系统的稳定性和承载负荷

水平拆分的缺点如下：

- (1) 表拆分后，数据分散存放，执行跨库的表连接（join）操作比较麻烦，而且性能也比较差
- (2) 数据扩容的难度和维护量极大。例如，增加了新的数据库实例，就要重新分布数据

13.1.2、垂直拆分：垂直拆分的每个子表都需要有主键

垂直拆分的优点如下：

- (1) 拆分后业务清晰，系统之间容易整合
- (2) 便于管理和维护数据

垂直拆分的缺点如下：

- (1) 部分表进行连接（join）困难，只能通过接口方式解决，增加了系统的复杂度
- (2) 存在单库性能瓶颈，不易进行单库的性能提升

13.1.3、为了对表进行精粒度的拆分，还可以对一张表同时进行垂直拆分和水平拆分。

13.2、shardingSphere简介

shardingSphere由Sharding-JDBC、Sharding-Proxy和Shardign-Sidecar（规划中）这三款各自独立的产品组成。

sharding产品	连接的数据库类型	连接消耗数	支持的语言	性能	无中心化	静态入口
sharding-JDBC	任意	高	Java	损耗低	是	无
sharding-Proxy	Mysql/postgresql	低	任意	损耗路高	否	有
sharding-Sidecar	Mysql/postgresql	高	任意	损耗低	是	无

13.2.1、Sharding-JDBC简介

sharding-JDBC定位为轻量级的分库分表中间件。具有以下适用性

- 可以对接任何基于Java的ORM框架，如JPA、Hiberanate、mybatis、Spring Data API、Spirng JDBC Template，或直接对接JDBC API。
- 可以对接任何第三方的数据库连接池，如DBCP、C3P0、BoneCP、Druid、HikariCP等
- 可以对接任何支持JDBC驱动的数据库，如mysql、oracle、sqlserver和postgresql

13.2.2、Sharding-Proxy简介

sharding-Proxy提供了数据库的代理。对于Shardign-JDBC，微服务通过Sharding-JDBC类库直连数据库；而对于Sharding-Proxy，微服务直连Sharding-Proxy服务器，然后由Shardign-Proxy服务器访问实际的数据库

13.3、shardingSphere的核心概念

shardingSphere围绕者分库分表，必须建立真实表和逻辑表在结构上的对应关系，还要建立真实表和逻辑表的记录的对应关系

13.3.1、与表有关的概念

与表相关的概念包括真实表、逻辑表、数据节点、绑定表和广播表

- (1) 真实表：数据库中真实存在的物理表，如Order_1表、order_2表
- (2) 逻辑表：存放同类型数据的所有真实表的合并表，如Order_1表和Order_2表的逻辑表为order，逻辑表是ShardignSphere提供的虚拟表，应该程序可以忽略分库分表的细节，只需操纵一张逻辑表
- (3) 数据节点：由数据源和真实表组成。例如，数据节点ds1.order_1表示位于ds1数据源的真实表order_1。
- (4) 绑定表：分片规则一致的关系表（主表、子表）
- (5) 广播表：在实际应用中，有些表没必要做分片，如字典表，因为它的数量不大，而且常常需要与存放海量数据的表进行关联查询，这种表称为广播表。广播表会在各个真实数据库中存储，并且表结构及表中的数据完全相同。

13.3.2、与分片相关的概念

向分表中存放数据时，需要遵循特定的策略，称之为分片策略。分片包含两层含义

- 对表进行水平拆分，使表中的数据可以分片存放到多个分表中
 - 对于应用程序提交的SQL语句，按照分片策略计算出SQL的路由，即到底在哪些真实表中执行SQL操作。
- (1) 分片键：表中用于分片的字段，即对表进行水平拆分的字段。例如，根据Order1订单表中的id主键分片，则id主键就是分片键。分片键分为两种。
 - 单分片键：把单个字段作为分片键
 - 复合分片键：把多个字段作为分片键
 - (2) 分片算法：由于分片算法和业务实现精密相关，因此并未提供内置的具体分片算法，而是从各种分片场景中抽象出更高层级的分片算法。目前提供4种分片算法。

分片算法	说明
精确分片算法	用单个字段作为分片键，分片操作为=、IN
范围分片算法	用单个字段作为分片键，分派操作为BETWEEN、AND、>、<、>=、<=
复合键分片算法	用多个字段作为分片键，分片逻辑较为复杂，需要应用开发者自行实现
Hint分片算法	在有些场景种，不是依据SQL语句中得到的分片键值进行分片，而是依据其他外置条件进行分片，就可以使用Hint分片算法。Hint分片算法支持Java API和SQL注释这两种使用方式

- (3) 分片策略：包含分片键和分片算法。分片策略结合分片键和分片算法，指定了具体的分片方式。目前包含5中分片策略。

表 13.3 分片策略的概述	
分片策略	说 明
标准分片策略 (StandardShardingStrategy)	最 基 本 的 SQL 语 句 中 <=、>、<、>=、<=、IN、BETWEEN 和 AND 的 分 片 策 略 支 持。StandardShardingStrategy 只 支 持 单 分 片 键。支 持 PrimaryShardingAlgorithm 和 RangeShardingAlgorithm 两 个 分 片 算 法。PrimaryShardingAlgorithm 是 基 础 的。用 户 还 可 选 择 RangeShardingAlgorithm。RangeShardingAlgorithm 是 最 先 进 的。用 户 可 选 择 BETWEEN、AND、>、<、>= 和 <= 分 片 操 作。如 果 不 能 选 择 RangeShardingAlgorithm，SQL 中 的 BETWEEN 和 AND 将 被 用 户 通 过 注 释 来 指 定。
复合分片策略 (ComplexShardingStrategy)	提 供 对 SQL 语 句 中 <=、>、<、>=、<=、IN、BETWEEN 和 AND 的 分 片 策 略 支 持。ComplexShardingStrategy 支 持 复 合 分 片 键。由 于 复 合 分 片 键 的 分 片 策 略 复 杂，因 此 并 未 进 行 过 多 的 研 究。而 说 表 理 解 复 合 分 片 策 略 复 杂 且 分 片 策 略 有 待 进 步 分 片 算 法。完 全 由 应 用 开 发 者 实 现。提 供 最 大 的 灵 活 性。
行表达式分片策略 (RuleShardingStrategy)	使 用 Guava 的 语 法。最 基 本 的 SQL 语 句 中 <=、>、<、>=、<=、IN、BETWEEN 和 AND 的 分 片 策 略 支 持。ComplexShardingStrategy 支 持 复 合 分 片 键。由 于 复 合 分 片 键 的 分 片 策 略 复 杂，因 此 并 未 进 行 过 多 的 研 究。而 说 表 理 解 复 合 分 片 策 略 复 杂 且 分 片 策 略 有 待 进 步 分 片 算 法。完 全 由 应 用 开 发 者 实 现。提 供 最 大 的 灵 活 性。
Hint 分片策略 (HintShardingStrategy)	通 过 Hint 算 法 指 定 分 片。用 户 从 SQL 语 句 中 提 取 分 片 值。
不分片策略 (NoneShardingStrategy)	不 进 行 分 片。

- (4) 分片策略维度：包括数据源维度和表分片维度。

13.4、shardingSphere的工作流程

应用程序向shardingSphere提交的SQL语句只会操纵逻辑表，而ShardingSphere会按照分片策略决定执行SQL语句的路由，即到底在哪些真实表中执行相应的操作。ShardingSphere的三个产品的主要工作流程是一致的。

- (1) SQL解析：SQL解析分为词法解析和语法解析。先通过词法解析器将SQL语法拆分为一个不可再分的单词，再使用语法解析器对SQL语句进行解析，并最终提炼出解析上下文。sharding-JDBC采用不同的解析器对使用各种数据库语言的

SQL语句进行解析。解析器类型包括mysql解析器、oracle解析器、sqlserver解析器、postgresql解析器和默认sql解析器。

- (2) 执行器优化：负责合并及优化分片条件
- (3) SQL路由：根据解析上下文即用户配置的分片策略生成路由，即到底在哪些数据节点中执行SQL操作
- (4) SQL改写：将SQL语句改写为在真实数据库中可以正确执行的语句。SQL语句改写分为正确性改写和优化改写
- (5) SQL执行：通过多线程执行器异步执行SQL语句
- (6) 结果归并：将多个数据节点中的执行结果归并以便于通过统一的JDBC接口输出

13.5. ShardingSphere的SQL规范

ShardingSphere对应用程序提交的SQL语句进行了一些语法限制。

13.5.1. 目前支持的SQL语法包括以下几种

- (1) 路由至单数据节点时，目前对mysql数据库完全兼容，对其他数据库的兼容还在完善中。
- (2) 路由至多数据节点时，全面支持DQL、DML、DDL、DCL、TCL，支持分页、去重、排序、分组、聚合、关联查询（不支持跨库关联）

13.5.2. ShardingSphere不支持的SQL语法包括以下几种

- (1) 路由至多数据节点时，不支持CASE WHEN、HAVING、UNION(ALL)
- (2) 对于子查询提供有限的支持，支持分页子查询。此外，无论嵌套多少层，ShardingSphere都可以解析至第一个包含逻辑表的子查询，一旦在下层嵌套中再次找到包含逻辑表的子查询，将直接抛出解析异常。
- (3) 由于归并的限制，目前不支持子查询中包含聚合函数（如sum()、avg()、max()和min()等）
- (4) 不支持包含schema的SQL，因为ShardingSphere的理念是像使用一个数据源一样使用多个数据源，所以SQL操作多是在同一个逻辑schema中。
- (5) 当分片键处于SQL的运算表达式或函数中时，ShardingSphere将采用全路由的形式获取结果。所谓全路由，是指路由到逻辑表的所有数据节点。

13.6. 运用Sharding-JDBC的范例

13.6.1. 加入Sharding-JDBC等的依赖

```
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.11</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-jdbc-core</artifactId>
  <version>5.3.2</version>
</dependency>
<dependency>
  <groupId>com.alibaba</groupId>
  <artifactId>druid-spring-boot-starter</artifactId>
  <version>1.1.17</version>
</dependency>
<dependency>
  <groupId>com.baomidou</groupId>
  <artifactId>mybatis-plus-boot-starter</artifactId>
  <version>3.5.3</version>
</dependency>
```

13.6.2. 创建拆分后的订单表

```
db_order
├── order_1
└── order_2
```

13.6.3. 配置Sharding-JDBC和Mybatis

```
server.port=9092
spring.application.name=order-service

spring.cloud.discovery.server-addr=127.0.0.1:8080

# ShardingSphere 配置
spring.shardingsphere.datasource.names=ds1

spring.shardingsphere.datasource.ds1.type=com.alibaba.druid.pool.DruidDataSource
spring.shardingsphere.datasource.ds1.driver-class-name=com.mysql.cj.jdbc.Driver
spring.shardingsphere.datasource.ds1.url=jdbc:mysql://127.0.0.1:3306/db_order?useSSL=false&serverTimezone=UTC
spring.shardingsphere.datasource.ds1.username=root
spring.shardingsphere.datasource.ds1.password=ysj1f

# 分表规则配置
spring.shardingsphere.sharding.tables.order.actual-data-nodes=ds1.order_${1..2}
spring.shardingsphere.sharding.tables.order.table-strategy.inline.sharding-column=id
spring.shardingsphere.sharding.tables.order.table-strategy.inline.algorithm-expression=order_${1..2} % 2 + 1

# 主键生成器配置
spring.shardingsphere.sharding.tables.order.key-generator.column=id
spring.shardingsphere.sharding.tables.order.key-generator.type=SNOWFLAKE

# SQL显示配置
spring.shardingsphere.props.sql.show=true

# Mybatis-Plus配置
mybatis-plus.configuration.map-underscore-to-camel-case=true
```

13.6.4. 创建Order实体类

```
@Data 9 个用法
@Table(name="`order`")
解释 | 添加注释 | X
public class Order {
    private Long id;
    private String userId;
    private String productCode;
    private BigDecimal money;
    private Integer count;
}
```

13.6.5. 创建访问数据的OrderDao接口

```
@Repository 2 个用法
解释 | 添加注释 | X
public interface OrderDao extends BaseMapper<Order> {
    解释 | 添加注释 | X
    // ...
    default List<Order> findAll(){ return this.selectList(new LambdaQueryWrapper<>());}
}
```

13.6.6. 创建OrderService类

DQL (数据查询语言) : select
DML (数据操纵语言) : insert, update, delete
DDL (数据定义语言) : create, alter, drop, truncate
DCL (数据控制语言) : grant, revoke
TCL (事务控制语言) : start transaction/begin, rollback, savepoint

```

@Transactional //生成订单分支事务 1个用法
public void create(String userId, String productCode, Integer count) {
    BigDecimal orderMoney = new BigDecimal(count).multiply(new BigDecimal(val.5));

    Order order = new Order();
    order.setUserId(userId);
    order.setProductCode(productCode);
    order.setCount(count);
    order.setMoney(orderMoney);

    orderDAO.insert(order);
    //修改账户余额分支事务
    accountFeignClient.debit(userId, orderMoney);
}

```

13.6.7. 创建OrderController类

```

@GetMapping("/create")
public Boolean create(String userId, String productCode, Integer count) {
    orderService.create(userId, productCode, count);
    orderService.create(userId: "USER_0001", productCode: "PRODUCT_0001", count: 1);
    return true;
}

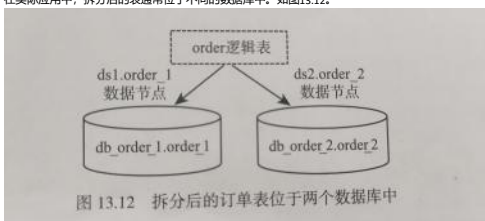
```

13.6.8. 访问127.7.1:9092/create, 可以看到数据库插入了一条新数据

id	user_id	product_code	count	money
2086283218285879298	USER_0001	PRODUCT_0001	1	5

13.6.9. 把订单表拆分到两个数据库中

在实际应用中, 拆分后的表通常位于不同的数据库中。如图13.12。



13.6.9.1. 在application.properties中配置ds1数据源和ds2数据源

13.6.10. 主表和子表为绑定表关系, 可以在application.properties文件中配置绑定关系

13.6.11. 配置广播表

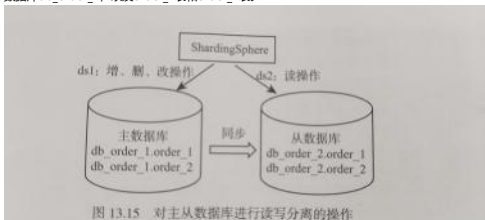
广播表属于系统中数据量小、更新少, 而且经常被联合查询的表。可以将广播表在每个数据库都保存一份, 对这类表的增、删、改操作会同时发送到所有分库执行, 从而确保广播表的数据在各个分库中保持一致

13.6.12. 主从数据库和读写分离

在并发环境中, 一个事务的更新操作会锁定订单数据, 导致其他事务的查询操作无法迅速返回查询结果。为了提高shardingSphere的并发性能, 可以在主从数据库的基础上进行读写分离

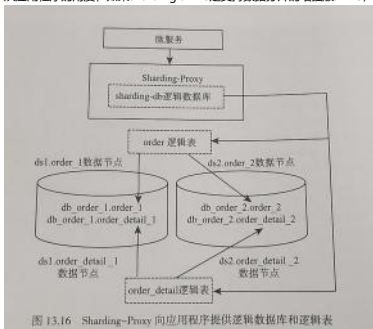
- 在主数据库中执行增、删、改等操作
- 在从数据库中执行查询操作

如图12.15. 两个数据库中数据的同步由数据库服务器的主从机制来保证, 而shardingSphere则负责实现读写分离。例如, 在主服务器上创建主数据库db_order_1, 以及order_1表和order_2表; 在从服务器中创建从数据库db_order_2, 以及order_1表和order_2表。



13.7. Sharding-Proxy简介

从应用程序的角度, 如果sharding-JDBC是支持数据分片的增强版JDBC, 那么Sharding-Proxy就是支持数据库分片的增强版数据库。应用程序可以像访问普通数据库一样访问Sharding-Proxy。



13.7.1. 安装和配置Sharding-Proxy

1. 启动Sharding-Proxy

为了使Sharding-Proxy为MySQL提供代理, 需要把MySQL的JDBC驱动类库mysql-connector-java.jar复制到Sharding-Proxy的lib目录下。运行bin/startup.bat文件, 就会启动Sharding-Proxy服务器

2. 基本配置

sharding-Proxy的conf目录下包含了所有的配置文件, 其中server.yaml用于配置基础信息, 如用户、口令、注册中心等。

3. 配置逻辑数据库和逻辑表

sharding-Proxy支持多逻辑数据库, 每个逻辑数据库的yaml配置文件的名字都以config-作为前缀。例如, config-sharding.yaml用于配置sharding_db逻辑数据库

13.7.2. 在微服务中访问Sharding-Proxy

1. 如果Shardign-Proxy为MySQL数据库提供代理, 并且微服务通过MyBatis访问数据库, 那么只需要加入MySQL的JDBC驱动依赖和MyBatis的依赖即可

2. 在application.properties文件中, 配置Sharding-Proxy的逻辑数据库sharding_db的数据源。

2.1. 如果没有显示设定数据源的类型, 将使用spring.datasource.type属性的默认值com.zaxxer.hikari.HikariDataSource。Hikari数据源的类库是由SpringBoot自动加入的, 因此无须再pom.xml我呢见中加入Hikari数据源的依赖。

如果要改为使用Druid数据源, 则需要加入以下代码spring.datasource.type=com.alibaba.druid.pool.DruidDataSource



压测（压力测试）：压测是模拟高并发/大流量场景, 对系统（接口/服务/数据库）施加远超日常峰值的负载, 验证系统的极限承载能力、稳定性、瓶颈点的测试方法。

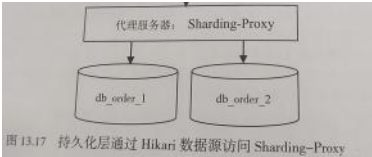


图 13.17 持久化层通过 Hikari 数据源访问 Sharding-Proxy

压测（压力测试）：压测是模拟高并发/大流量场景，对系统（接口/服务/数据库）施加远超日常峰值的负载，验证系统的极限承载能力、稳定性、瓶颈点的测试方法。
常用压测工具有：JMeter、Postman

13.7.3、配置影子数据源

在分布式的微服务系统中，软件架构非常庞大，业务模块又被切分得很细，这对于测试，尤其是**压力测试（简称压测）**带来了很大难度。如果把测试环境部署得和生产环境一模一样，就非常浪费资源。而如果只部署少量的服务，又不能进行全链路的整体压测。解决方案是在生产环境中直接进行压测，得出的结果也是真实有效的。但是如果不对压测数据进行特殊的处理，就会和生产环境中的真实数据混在一起。

为了把压测数据和真实数据分开，就需要配置影子数据源。Sharding-Proxy会根据配置的匹配规则，把压测数据分配到影子数据源中，从而与生产环境中的真实数据隔离开。

13.8、小结

- 为了简化应用程序的开发，shardingSphere作为介于应用程序和数据库之间的中间件，能够按照配置的片策略，决定执行SQL的路由。应用程序只需操纵ShardingSphere提供的逻辑表。在逻辑表中包含所有分表的数据
- shardingSphere按照实现方式的不同，包括Sharding-JDBC、Sharding-Proxy和Sharding-Sidecar这三款各自独立的产品。其中，Sharding-JDBC属于轻量级的Java类库，在Java的JDBC层提供数据库访问服务，以jar包形式使用。shrdign-Proxy通过封装了数据库协议的独立Sharding-Proxy服务器，为原始数据库提供透明化的代理，可以支持各种开发语言。Sharding-Sidecar还在规划中。
- shardingSphere不仅可以配置分片策略，还可以指定表之间的绑定关系，以及进行读写分离、对数据加密和操纵影子数据源。

14、分布式缓存数据库：Redis

频繁像磁盘读写大量数据会影响程序的运行性能。可以在内存中开辟一块缓存，把经常读写的数据放在缓存中，这样就能减少读写磁盘的次数。

如果缓存是由应用程序负责创建的，那么它位于应用程序所在进程的内存空间内，就不能被其他应用程序访问。所以在分布式系统中，可以使用专门的缓存数据库redis。每个微服务都可以通过Redis客户端访问Redis服务器，向Redis服务器读写数据。

14.1、Redis简介

redis是一个用C语言开发的开源的缓存数据库。具有以下优点

- 以key/value的形式存取数据
- 读写性能极高。目前redis读的速度是110000次/s，写的速度是81000次/s。
- 支持事务的原子性。允许把一组操作作为一个事务，保证它的原子性，即所有的操作执行成功才提交事务，否则就撤销事务。
- 支持丰富的数据类型。Redis支持的数据类型包括string（字符串）、list（列表）、set（集合）、zset（有序集合）、hash（哈希映射，也称为哈希散列表）

reis不可以取代MySQL等数据库服务器。因为MySQL等数据库服务器允许客户程序通过SQL语句对数据库进行灵活的增删、改和查询操作。相比之下，Redis提供的操作持久化数据的能力是有限的。

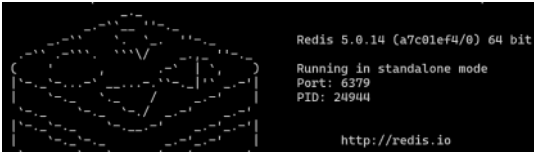
Redis最显著的优点是在读写缓存数据方面具有卓越的性能。

以下是适合存放到redis中的数据

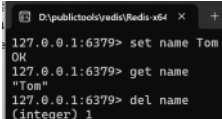
- 字典表、配置数据。这类数据尽管改动很大，但是会被频繁的查询
- 热点业务数据。这类数据的数据量不是很大，但是会被频繁地访问
- 临时数据。应用程序临时处理地数据，不需要长久保存，如会话范围内地数据，会话结束后，这些数据就会结束生命周期

14.2、安装和启动redis

下载Redis的zip压缩包并解压。运行根目录下的redis-server.exe程序就会启动Redis服务器，默认情况下监听6379端口。



运行Redis目录下的redis-cli.exe程序，就会启动基于DOS命令行窗口的Redis客户端程序

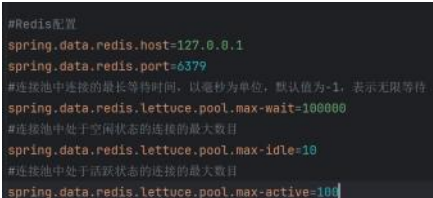


14.3、在微服务中访问redis

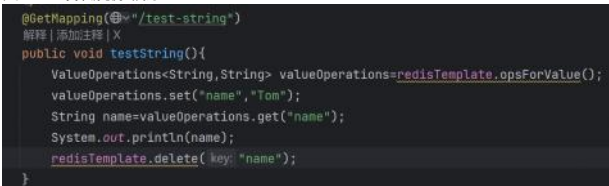
在order-service的pom.xml文件中加入Redis客户端组件的依赖



在order-service的application.properties中配置Redis



在controller类中编写测试代码



14.4、Redis数据类型

Redis服务器中存放的数据为Redis数据类型，包括以下5种数据类型

- string类型：Redis最基本的数据类型。在一对key/value数据种，作为value的string类型数据的最大容量是512MB.string类型是二进制安全的，意味着可以包含二进制数据，如jpg图片或者对象的序列化数据。
因此，Redis的string类型与Java的String类型不是对等的。
- list类型：元素按照索引位置排序的列表。数据结构和java.util.List列表相似
- set类型：不存在重复的集合，数据结构和java.util.Set集合相似
- zset类型：对元素按照评分排序的集合
- hash类型：包含映射键/映射值得哈希映射，也称为哈希散列表，数据结构和java.util.Map映射相似

向Redis服务器种存入一堆key/value数据时，key只能是string类型，value可以是这5种类型

14.4.1、读写list类型数据

ListOperations接口用于操作列表，包括以下方法，其中K和V是Java泛型类：

- leftPush (K key, V value)：从列表的左边加入元素
- rightPush (K key, V value)：从列表的右边加入元素
- size (K key)：返回与key对应的列表种的元素的数目
- range (K key, long start, long end)：从列表种读取特定范围内的元素。其中，start指定起始索引；end指定结束索引。列表中第一个元素的索引为0，第二个元素的索引为1，以此类推。当start参数为0，end参数为-1时，表示读取列表中的所有元素。

```

@GetMapping("@*/test-list")
// 解释 | 添加注释 | X
public void testList(){
    ListOperations<String,Integer> listOperations=redisTemplate.opsForList();
    listOperations.leftPush("nums",200);
    listOperations.leftPush("nums",100);
    listOperations.rightPush("nums",300);
    listOperations.rightPush("nums",400);
    List<Integer> nums=listOperations.range( key: "nums", start: 0, end: -1);
    for (Integer num : nums) {
        System.out.println(num);
    }
    redisTemplate.delete( key: "nums");
}
}

```

在test List () 方法的程序代码中，存入列表的元素的Java类型为Integer；而在Redis服务器中，存放的是他们的序列化数据，为string类型。

14.4.2. 读写set类型数据

SetOperations接口用于操作集合，包括以下方法。

- add (K key, V value) : 向集合中加入一个或多个元素
- remove (K key, Object... values) : 从集合中删除一个或多个元素
- size (K key) : 返回与key对应的集合中元素的数量
- intersect (K key, K otherK) : 返回两个集合的交集

```

@GetMapping("@*/test-set")
// 解释 | 添加注释 | X
public void testSet(){
    SetOperations<String,String> setOperations=redisTemplate.opsForSet();
    setOperations.add( key: "users", ..values: "red");
    setOperations.add( key: "colors", ..values: "white");
    setOperations.add( key: "colors", ..values: "blue");
    for (String color : setOperations.members( key: "colors")) {
        System.out.println(color);
    }
    redisTemplate.delete( key: "colors");

    setOperations.add( key: "user1:hobbies", ..values: "旅游","跳舞","绘画");
    setOperations.add( key: "user2:hobbies", ..values: "旅游","唱歌");
    setOperations.add( key: "user3:hobbies", ..values: "打篮球","看电影","绘画");
    Set<String> hobbies = setOperations.intersect( key: "user1:hobbies", otherKey: "user2:hobbies");
    for (String hobby : hobbies) {
        System.out.println("user1和用户2的共同兴趣: "+hobby);
    }
    hobbies=setOperations.intersect( key: "user1:hobbies", otherKey: "user3:hobbies");
    for (String hobby : hobbies) {
        System.out.println("user1和用户3的共同兴趣: "+hobby);
    }
    redisTemplate.delete( key: "user1:hobbies");
    redisTemplate.delete( key: "user2:hobbies");
    redisTemplate.delete( key: "user3:hobbies");
}
}

```

结果

```

white
blue
user1和用户2的共同兴趣: 旅游
user1和用户3的共同兴趣: 绘画

```

14.4.3. 读写zset类型数据

zsetOperations接口用于操作zset集合，包括以下方法。

- add (K key, V value, double score) : 向集合中加入一个元素，参数score表示评分。
- range (K key, long start, long end) : 读取集合中特定范围内的元素。其中，start参数指定起始索引；end参数指定结束索引。zset集合中第一个元素的索引为0，第二个元素的索引为1，一次类推。当start参数为0，end参数为-1时，表示读取集合中的所有元素。
- reverseRange (K key, long start, long end) : 读取集合按照倒序排序后特定范围内的元素。
- rangeByScore (K key, double min, double max) : 读取集合中特定范围内的元素。其中，min参数指定最小评分；max参数指定最大评分
- reverseRangeByScore (K key, double min, double max) : 读取集合按照倒序排序后特定范围内的元素。其中，min参数指定最小的评分；max参数指定最大的评分。
- size (K key) : 返回与key对应的zset集合中元素的数量
- incrementScore (K key, V value, double delta) : 增加一个元素的评分值。delta参数表示评分的增加值。

```

@GetMapping("@*/test-zset")
// 解释 | 添加注释 | X
public void testZset(){
    ZSetOperations<String,String> zSetOperations=redisTemplate.opsForZSet();
    zSetOperations.add( key: "article", value: "about java", score: 1);
    zSetOperations.add( key: "article", value: "about redis", score: 2);
    zSetOperations.add( key: "article", value: "about Linux", score: 1);
    zSetOperations.add( key: "article", value: "about springcloud", score: 4);
    System.out.println("刷新前的排行榜，由高到低");
    Set<String> articles=zSetOperations.reverseRange( key: "article", start: 0, end: -1);
    for (String article : articles) {
        System.out.println(article);
    }
    zSetOperations.incrementScore( key: "article", value: "about java", delta: 5);
    System.out.println("刷新后的排行榜，由低到高");
    for (String article : zSetOperations.range( key: "article", start: 0, end: -1)) {
        System.out.println(article);
    }
    redisTemplate.delete( key: "article");
}
}

```

结果

```

刷新前的排行榜，由高到低
about springcloud
about redis
about Linux
about java
刷新后的排行榜，由低到高
about Linux
about redis
about springcloud
about java

```

14.4.4. 读写hash类型数据

HashOperations接口用于操作哈希映射，包括以下方法

- put (H key, HK hashKey, HV value) : 加入一对映射键/映射值。其中hashKey参数表示映射键；value参数表示映射值。
- entries (H key) : 返回与key参数对应的哈希映射，返回值为Map<HK,HV>类型
- keys (H key) : 返回与key参数对应的哈希映射的所有哈希映射键的集合，返回值为Set<HK>类型
- get (H key, Object hashKey) : 返回与key参数对应的哈希映射中与hashKey映射键对应的映射值。

- size (K key)：返回与key对应的哈希映射中元素的数目

14.4.5. 序列化Java对象

当应用程序写数据时，Java对象被Redis的序列化器转换为序列化数据，再存放至Redis服务器中。当应用数据读数据时，Redis服务器中的序列化数据被反序列化为Java对象

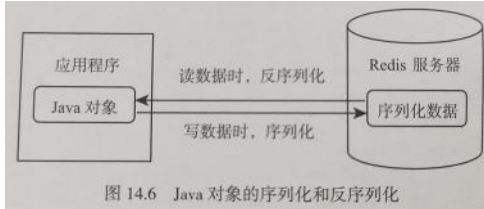


图 14.6 Java 对象的序列化和反序列化

14.5. Redis集群

为了保证Redis的高可用性，可以为Redis建立集群，具体包括以下三种模式

- 主从模式：主节点（Master）会自动将数据同步到从节点（Slave），可以进行读写分离
- 哨兵模式：再主从模式的基础上增加了哨兵工具（sentinel），会监控每个节点的运行状态。当主节点出现故障宕机时，该模式会把哨兵就像兢兢业业的哨兵一样，实时监控请求流量的变化，因此业界也把Sentinel称为哨兵工具
- 集群模式：采用分布式的存储，数据存储到多个节点

14.5.1. 主从模式

在主从模式下，Redis主节点和从节点中的数据保持一致

Redis主节点和从节点的数据的同步流程如下。其中步骤（1）~步骤（4）是从节点的初始化过程，步骤（5）~步骤（6）是从节点在运行期间的常规同步操作。

- （1）从节点连接主节点，发送SYNC同步命令
- （2）主节点接收到SYNC命令后，先执行BGSAVE命令，把缓存中的数据持久化到RDB文件中，接着利用缓冲区记录此后执行的所有写命令。
- （3）主节点执行完BGSAVE命令后，向所有从节点发送快照文件，并在发送期间继续记录所执行的写命令。快照文件包含了主节点当前的存储数据
- （4）从节点收到快照文件后丢弃所有旧数据，载入收到的快照，完成初始化
- （5）主节点发送完快照文件后开始向从节点发送缓冲区中记录的写命令
- （6）主节点每执行一个写命令，就会向从节点发送相同的写命令，从节点接收并执行收到的写命令

主从模式具有以下优点

- 主节点会自动将数据同步到从节点，可以进行读写分离。例如，应用程序对主节点执行写操作，对从节点执行读操作
- 从节点同样可以接收其他从节点的连接和同步请求，这样可以有效地分担主节点地同步压力
- 主节点以非阻塞地方式与从节点进行同步，所以主从节点同步期间，客户端仍然可以向主节点提交查询或修改请求
- 从节点也以非阻塞地方式完成数据同步。在同步期间，如果有客户端向从节点提交查询请求，则从节点返回同步之前的数据

主从模式具有以下缺点

- 不具备自动容错和恢复功能，主从节点的宕机都会导致客户端的部分读写请求失败
- 在主节点宕机前，如果有部分数据未能即使同步到从节点，则会导致数据不一致问题
- 较难支持在线扩容，在集群容量达到上限时，在线扩容会变得很复杂

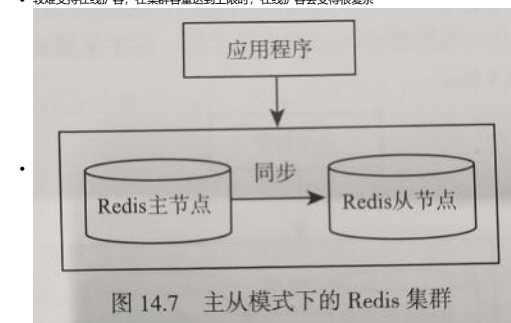


图 14.7 主从模式下的 Redis 集群

14.5.2. 哨兵模式

在主从模式中，当主节点宕机后，可以将以一个从节点转换为主节点，以便继续提供服务，但是这个转换过程需要用户手动进行操作。为了更方便管理Redis集群，Redis提供了哨兵工具来实现自动化的系统监控和故障恢复功能。

在主从模式的基础上增加哨兵的功能，就形成了哨兵模式

- （1）哨兵工具具有以下功能
 - 监控主节点和从节点是否正常运行
 - 当主节点出现故障时自动将从节点转换为主节点
- （2）哨兵工具作为独立的进程运行，监控Redis集群的工作方式如下：
 - 每个哨兵进程以每秒一次的频率向整个集群中的主节点及其他哨兵进程发送一个PING命令。集群中素有的主节点、从节点和哨兵进程均作为被监控的实例
 - 如果一个实例距离最后一次有效回复PING命令的时间超过down-after-milliseconds选项所指定的值，那么该实例会被哨兵进程标记为主管下线（SDOWN）
 - 如果一个主节点被标记为主管下线，那么正在监视该主节点的所有哨兵进程会以每秒一次的频率来确认主节点的确进入了主管下线状态
 - 当有足够数量的哨兵进程（大于等于配置文件指定的值）在指定的时间范围内确认主节点进入了主管下线状态时，主节点会被标记为客观下线（ODOWN）。
 - 在一般情况下，每个哨兵进程会以每10s一次的频率向集群中所有主从节点发送INFO命令。当主节点被哨兵进程标记客观下线时，哨兵进程向下线的主节点的所有从节点发送INFO命令的频率会从每10s一次改为每秒一次
 - 如果没有足够数量的哨兵进程同意主管节点下线，那么主节点的客观下线状态就会被移除
- （3）哨兵模式的优点如下：
 - 基于主从模式，因此具有主从模式的所有优点
 - 从节点可以自动转换为主节点，系统更健壮，可用性更高
- （4）哨兵模式的缺点时较难支持在线扩容，当集群容量达到上限时，在线扩容会变得很复杂

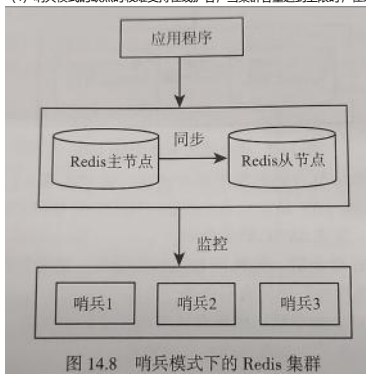


图 14.8 哨兵模式下的 Redis 集群

14.5.3. 集群模式

哨兵模式已经实现了高可用和读写分离，但是在这种模式下，每台Redis节点都存储相同的数据，因此浪费内存且扩容很困难，为了解决这一问题，在Redis3.0以上版本中加入了集群（cluster）模式，以实现分布式存储，也就是可以把数据分布式存储到多个Redis节点中。

- （1）集群模式无中心结构，它的特点如下：
 - 所有Redis节点彼此互联，内存使用二进制协议优化传输速度和带宽
 - 认定一个节点下线的标准是集群中超过半数的节点对其检测的结果为不可用
 - 客户端与Redis节点直连，不需要中间代理层。客户端不需要连接集群中的所有节点，连接集群中任何一个可用节点即可，每个节点上都有一个集群管理组件，它会计算到底把数据存储到哪个节点中
 - 支持在线扩容，允许向运行中的就请你增加新的节点
- （2）为了保证高可用和健壮性，集群模式也建立在主从模式的基础上，一个主节点对应一个或多个从节点，当主节点宕机后，就会启动从节点。

14.5.4. 搭建Redis集群

14.5.1、如图14.9，一共建立6个节点，有3个是主节点，另外3个是从节点

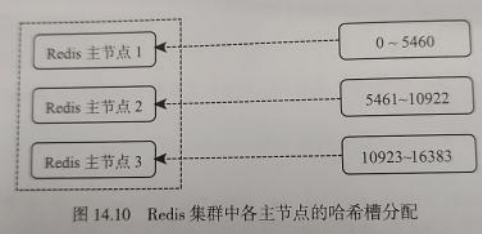
- （1）在redis1安装目录下创建redis.conf文件，配置监听端口、开启集群功能等

```
port 7001
#开启集群功能
cluster-enabled yes
#集群的配置文件名称不需要自己创建，而在redis自己维护
cluster-config-file node-7001.conf
#节点心跳丢失的超时时间
cluster-node-timeout 5000
#持久化文件存储目录
dir D:\publictools\redis\data
#绑定地址
bind 0.0.0.0
#是否让redis后台运行
daemonize no
#保护模式
protected-mode no
#数据库数量
databases 1
```

- (2) 在redis1目录下创建data目录，Redis节点的存储数据以及node-7001.conf集群配置文件都会存放在该目录下
- (3) 把Redis1目录复制为redis2、redis3、redis4、redis5、redis6，并修改redis.conf下的端口和node-700*.conf配置
- (4) 启动6个Redis服务，运行以下命令。

```
D:\publictools\redis\redis1>redis-cli --cluster create --cluster-replicas 1 127.0.0.1:7001 127.0.0.1:7002 127.0.0.1:7003
127.0.0.1:7004 127.0.0.1:7005 127.0.0.1:7006
>>> Performing hash slots allocation on 6 nodes...
Master[0] -> Slots 0 - 5460
Master[1] -> Slots 5461 - 10922
Master[2] -> Slots 10923 - 16383
Adding replica 127.0.0.1:7005 to 127.0.0.1:7001
Adding replica 127.0.0.1:7006 to 127.0.0.1:7002
Adding replica 127.0.0.1:7004 to 127.0.0.1:7003
>>> Trying to optimize slaves allocation for anti-affinity
[WARNING] Some slaves are in the same host as their master
M: fa767d905bdfb990537344df96ce520095dca82 127.0.0.1:7001
slots:[0-5460] (5461 slots) master
M: 4bf8c5063b95fa19e76feec49df1650785c9cc3 127.0.0.1:7002
slots:[5461-10922] (5462 slots) master
M: 06f082f8e2fcf507fab90325e0121893f377250d 127.0.0.1:7003
slots:[10923-16383] (5461 slots) master
S: 628c8252745f61c15ddb258ada73ccf3c71bdd36 127.0.0.1:7004
replicates fa767d905bdfb990537344df96ce520095dca82
S: 831f07197c3ebb37ed6bd7ce9c355ec4f4f76c6fb 127.0.0.1:7005
replicates 4bf8c5063b95fa19e76feec49df1650785c9cc3
S: 754d6aac1f773f60d73224e0917694ff5a88c91e 127.0.0.1:7006
replicates 06f082f8e2fcf507fab90325e0121893f377250d
Can I set the above configuration? (type 'yes' to accept): yes
>>> Nodes configuration updated
>>> Assign a different config epoch to each node
>>> Sending CLUSTER MEET messages to join the cluster
```

14.5.2. 向Redis集群中存入一对key/value，这个key/value会存放到特定的哈希槽中，买个Redis主节点包含如果哈希槽（hash slot），Redis集群中内置了16384个哈希槽，分摊到所有的Redis主节点中，在图14.10中，3个Redis主节点分别分配了5461、5462和5463个哈希槽。



14.6. 在微服务中访问Redis集群

14.3节已经在order-service中访问哪个Redis服务器，如果要改为访问Redis集群，只需修改application.properties配置文件即可。Lutture客户端组件会建立与集群中Redis节点连接

```
#Redis配置
spring.data.redis.cluster.nodes=127.0.0.1:7001,127.0.0.1:7002,127.0.0.1:7003,127.0.0.1:7004,127.0.0.1:7005,127.0.0.1:7006
spring.data.redis.cluster.max-redirects=6
#连接池中连接的最长等待时间，以毫秒为单位，默认为-1，表示无限等待
spring.data.redis.lettuce.pool.max-wait=100000
#连接池中处于空闲状态的连接的最大数目
spring.data.redis.lettuce.pool.max-idle=10
#连接池中处于活跃状态的连接的最大数目
spring.data.redis.lettuce.pool.max-active=100
```

14.7. 小结

- (1) 在分布式的微服务中，服务可以分布式部署，需要永久存储的业务数据可以分布式存储，不仅如此，缓存中的数据也可以分布式存储。本章介绍了分布式缓存数据库Redis的用法，Redis以key/value的形式存储数据
- (2) Redis的数据类型包括5种：字符串（string）类型、列表（list）类型、集合（set）类型、有序集合（zset）类型和哈希（hash）类型。Redis以key/value的形式存储数据时，key只能是string类型，value可以是5种类型之一，当Java程序把自定义的User对象存储到Redis中时，需要先由Redis的序列化器进行序列化，得到string类型的数据再进行存储
- (3) Redis的集群包括3种模式：主从模式、哨兵模式和集群模式，主从模式可以保证Redis的高可用性，但是当主节点宕机后，从节点不能自动转换为主节点。哨兵模式建立在主从模式的基础上，通过哨兵工具的监控自动进行从节点到主节点的转换，集群模式也建立在主从模式的基础上，并且能够把数据分布存储到多个节点中。

15. 分布式任务调度框架：XXL-JOB

- (1) XXL-JOB框架主要包括以下两个角色

- xxl-job-admin调度中心：统一管理执行器和各个任务，指派执行器执行特定任务
- xxl-job-executor执行器：执行特定的任务，无论任务执行成功还是失败，都会向调度中心返回执行结果

- (2) XXL-JOB调度的定时任务是指在约定时间执行的一段程序代码，定时任务适用于以下场景

- 批量处理统计数据的场景。例如，批量统计上个月的订单数据
- 时间驱动的场景。例如，在某个时间点群发短信或邮件
- 固定频率的场景。例如，每隔5min需要执行某种业务操作

- (3) XXL-JOB的调度中心和执行器是独立的进程，任务的程序代码可以部署在执行器中，也可以由调度中心托管，调度中心和执行器都可以建立集群，这样就形成了分布式的任务调度系统

- (4) 调度中心的调度信息存放在数据库中，系统管理员通过调度中心的管理界面，可以方便地对调度信息进行添加、更新删除和查询

15.1. 安装和运行XXL-JOB

XXL-JOB是一个spring项目，下载XXL-JOB并用IDEA打开xxl-job-master，其中包含三个模块：

- xxl-job-admin模块：XXL-JOB的调度中心
- xxl-job-core模块：XXL-JOB的核心模块，提供了XXL-JOB的类库，其他模块都依赖该模块
- xxl-job-executor-samples模块：提供了XXL-JOB执行器的范例。它的xxl-job-executor-sample-springboot子模块适用了Spring Boot框架

15.1. 创建XXL-JOB的数据库

XXL-JOB会把调度信息存放在数据库中，doc/db/tables_xxl_job.sql文件中包含了创建xxl_job数据库、表及向表中插入初始数据的SQL语句，在MySQL中运行tables_xxl_job.sql文件中的SQL语句。会创建8个表，各个表的作用如下。

表名	表 说	包含内容
xxl_job_group	执行器信息表	包含执行器基础信息
xxl_job_info	调度任务信息表	包含调度任务的基础信息，如任务分组、任务名称、调度策略、执行时间、执行人等
xxl_job_lock	任务调度锁表	包含调度锁的信息
xxl_job_log	调度日志表	包含调度日志的基础信息，如调度结果、执行时间、调度人、调度策略和调度时间
xxl_job_log_report	调度日志聚合表	包含调度日志的聚合信息
xxl_job_logglue	GLUE任务更新日志表	包含GLUE任务更新日志，对GLUE任务进行版本更新和回滚的日志
xxl_job_registry	执行器注册表	包含执行器注册信息和调度中心的基础信息
xxl_job_user	系统用户表	包含调度中心的用户信息，表中有一行初始记录，用户名为admin，密码为123456

调度中心支持集群部署，集群中的各节点需要连接同一个MySQL实例，以保证调度信息的一致与同步。如果MySQL数据库包含了主从节点的部署，调度中心集群的各节点必须连接数据源主节点。

15.1.3. 运行和访问XXL-JOB调度中心

运行xxl-job-admin模块的自动类，访问8080/xxl-job-admin，就会登录到调度管理页面。



15.1.4. 配置XXL-JOB执行器
xxl-job-executor-sample-springboot模块是XXL-JOB执行器的范例，执行器依赖xxl-job-core模块。启动范例的主类。就会启动XXL-JOB执行器，该执行器会注册到调度中心

AppName	名称	注册方式	OnLine 机器地址	操作
xxl-job-executor-sample	示例执行器	自动注册	查看 (1)	操作

- 15.2. 创建和执行GLUE模式的任务
- 任务的具体操作是由一段程序代码来指定的。XXL-JOB可以通过以下模式创建任务
- GLUE模式：在调度中心的GLUE IDE界面中创建GLUE模式的任务（简称GLUE任务），任务代码由调度中心托管。GLUE模式支持的语言包括Java、python、php、Node.js等。这种模式非常轻量，便于在线维护，但是编写复杂的业务逻辑比较麻烦
 - BEAN模式：在执行器的模块中，用@xxlJob注解定义任务代码。这种模式可以编写任意的复杂业务逻辑

创建和执行GLUE任务的步骤如下：

- (1) 在调度中心的管理界面选择菜单“任务管理”->“新增”，新增一个GLUE任务。

新增

基础配置

执行器*

示例执行器

任务描述*

请输入任务描述

负责人*

请输入负责人

报警邮件*

请输入报警邮件，多个邮件地址则逗号分隔

调度配置

调度类型*

CRON

Cron*

cron表达式...

任务配置

运行模式*

BEAN

JobHandler*

请输入JobHandler

任务参数*

请输入任务参数

高级配置

路由策略*

第一个

子任务ID*

请输入子任务的任务ID,如存在多个则逗号分隔

调度过期策略*

忽略

阻塞处理策略*

单机串行

任务超时时间*

任务超时时间，单位秒，大于零时生效

失败重试次数*

失败重试次数，大于零时生效

保存

取消

- (2) 新增后，选择菜单“操纵”，选择GLUE IDE。

操作

执行一次

查询日志

注册节点

下次执行时间

GLUE IDE

停止

编辑

删除

复制

- (3) 在编辑器中的编写逻辑

```
package com.xxl.job.service.handler;

import com.xxl.job.core.context.XxlJobHelper;
import com.xxl.job.core.handler.IJobHandler;

public class DemoGlueJobHandler extends IJobHandler {

    @Override
    public void execute() throws Exception {
        System.out.println("XXL-JOB, Hello Tom.");
        XxlJobHelper.log("XXL-JOB, Hello World.");
    }
}
```

- (4) 启动后，点击菜单“查询日志”，显示每次执行任务的日志清单。

2	2026-01-01 13:14:40	失败	查看
2	2026-01-01 13:14:38	失败	查看

15.3. 创建和执行BEAN模式的任务

BEAN模式的任务是由xxl-job-executor-sample-springboot执行器模块中的Bean实例实现的。任务的源代码由执行器模块来维护。调度中心负责调度任务，执行器负责执行任务。创建和执行BEAN模式的任务的步骤如下：

- (1) 在执行器模块定义方法，它用@xxlJob（“demoJobHandler”）注解标识，表明该方法定义了一个任务，参数demoJobHandler代表任务的名字
- (2) 在调度中心的管理页面中新增一个任务，运行模式改为BEAN，JobHandler设为demoJobHandler。

15.3.1. 任务的初始化和销毁

一个任务的生命周期包括以下三个阶段

- (1) 初始化：当用户在调度中心的管理页面启动任务时，任务就会进入初始化阶段
- (2) 运行：当调度中心指派执行器定时执行任务时，任务就会进入运行阶段
- (3) 销毁：当用户在调度中心的管理页面中停止任务时，任务就会进入销毁阶段

在任务的初始化和销毁阶段，也可以自定义一些与特定业务相关的操作。例如，@xxlJob（value=“demoJobHandler”，init=“init”，destory=“destory”）

- 15.3.2. 任务也可以带参数。在执行器模块的犯法中，可以通过getUobParam（）方法来获取参数。参数可以在调度中心的管理页面中新增一个任务，指定任务参数。

15.4. 执行器集群和分片执行任务

如果任务繁重，可以对执行器建立集群。调度中心会调度集群中的各个执行器节点分摊执行任务。搭建执行器集群及分片执行任务的步骤如下：

- (1) 在执行器模块的方法中，可以通过getShardIndex（）方法得到当前执行该方法的执行器节点的分片序号，通过getShardTotal（）方法得到分片总数。

(2) 在IDEA中创建两个执行器的自动项，server.port和xxl.job.executor.port属性分别设为8082和8889，使用创建的自动配置运行该类，这样就启动了两个执行器节点，他们监听不同的窗口。

(3) 在调度中心得管理页面中新建一个分片执行得任务，把“路由策略”设为“分片广播”

15.5、搭建调度中心得集群

调度中心也可以建立集群。只要复制调度中心得模块，每个模块得配置文件需要做一些修改，保证每个调度中心节点的地址不同，但连接的数据库相同，并且服务的时钟保持一致。还可以使用Nginx为调度中心提供统一的代理，在Nginx的conf/nginx配置文件中需要设置代理的调度中心节点的地址，以及代理转发的URL。在执行器模块的配置中，只需指定Nginx的地址。

15.6、小结

XXL-JOB 是一个分布式、轻量级的任务调度框架，没有开发门槛：学习简单、部署简单、易扩展的架构。XXL-JOB 语言跨语言中心来调度多个执行器并行任务，利用 MySQL 等数据库来存储调度信息。任务包括（Cront 表达式和 DSL 脚本，以及分布式任务列表代码由调度中心负责，支持多种开发语言；XXL-JOB 模式自身自带的代码执行逻辑，支持 Java 语言。

XXL-JOB 提供了操作方便的管理界面，为任务执行失败时，还能发送告警邮件。为了分散执行器运行负载，可以建立执行器集群，为任务执行任务，为了保证调度中心的可用性，也可以建立调度中心的集群。

至此，整个 Spring Cloud Alibaba 框架中的各种组件都一一亮相，展示了它们的功能力。运行部署稍周详，不能把那个部署器多使用的一个按钮，做一个部署的按钮。

- Gateway 就像个门岗，管理着所有的入口，进行业务安全的，掌握进入系统的人流。
- 各种微服务提供端地址的列表管理，查看商品、朋友、餐饮等等，然后为设备提供数据。
- Nacos 注册中心就像（114 查询系统，各种门店都在（114 查询系统中做了登记，消费者只要查一下（114 查询系统，就能知道各门店的具体地址。
- 各种门岗之间的业务数据，如设备信息会从数据库读取，OpenFeign 和 Dubbo 微服务调用，既有消费者的请求发送给提供者，再把提供者的响应发送给消费者。
- 连接上在网关侧（114 系统，Sentinel 故障处理系统，负责管理流量，解决过载问题。
- 各种门岗之间不仅依靠业务数据，还需要异步通信，Stream 和 RocketMQ 就是支持异步通信的数据库系统，为业务逻辑的提供数据在数据库中的，消费者从数据库的数据库数据。
- SkyWalking 就像运维上的监控系统，通过在每个节点安装探针，监控整个系统的运行状况，及时发现故障。
- 微服务就像数据库里的仓库，如果数据量很大，可以将某些数据放到多个仓库的仓库，ShardingSphere 就像数据库，封装了数据库的抽象，为所有的数据库提供业务逻辑统一的数据视图。
- 非多个版本的数据集中同时的更新时，为了保证事务的 ACID 特性，引入了 Seata 协调器，统一管理分布式事务。
- 仓库除了个地址的编码规则，客户向仓库再取数据比较耗时，Redis 数据位于十多个的缓存中心的小缓存，客户向仓库再取数据很快，但是它的容量比数据库的仓库小得多。
- 仓库里还有一些安全防护，如定时清理数据，安全防护，防止入口等，XXL-JOB 就像调度器，负责解耦和执行门岗的执行任务。